Pashov Audit Group

Regnum Aurum Security Review

Conducted by:
t.aksoy
jesjupyter
Said
ast3ros

June 26th 2025 - July 12th 2025

Contents

1. About Pashov Audit Group

Pashov Audit Group consists of 40+ freelance security researchers, who are well proven in the space - most have earned over $100k in public contest rewards, are multi-time champions or have truly excelled in audits with us. We only work with proven and motivated talent.

With over 300 security audits completed — uncovering and helping patch thousands of vulnerabilities — the group strives to create the absolute very best audit journey possible. While 100% security is never possible to guarantee, we do guarantee you our team's best efforts for your project.

Check out our previous work here or reach out on Twitter @pashovkrum.

2. Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs and on-chain monitoring are strongly recommended.

3. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users
- **Medium** - leads to a moderate material loss of assets in the protocol or moderately harms a group of users
- **Low** - leads to a minor material loss of assets in the protocol or harms a small group of users

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive

4. About Regnum Aurum

Regnum Aurum (RAAC) is a fractionalization platform that tokenizes real estate into NFTs (RAACNFT) and fractional index tokens (iRAAC), enabling on-chain lending, borrowing, and liquidity against property value. By combining Chainlink-powered appraisals, a hybrid RWA Vault, and veRAAC governance the protocol enables programmable debt positions against real estate assets with on-chain liquidation mechanisms.

5. Executive Summary

A time-boxed security review of the **RegnumAurumAcquisitionCorp/core** repository was done by Pashov Audit Group, during which **t.aksoy**, **jesjupyter**, **Said**, **ast3ros** engaged to review **Regnum Aurum**. A total of **77** issues were uncovered.

Protocol Summary	
Project Name	Regnum Aurum
Protocol Type	RWA tokenization
Timeline	June 26th 2025 - July 12th 2025

Review commit hash:
- 167c4ed734000a717fee62e1709cb08ed419c1f2 (RegnumAurumAcquisitionCorp/core)

Fixes review commit hash:
- 5fb3f5687237c6a8b527876087e7c1f36ba6bd3b (RegnumAurumAcquisitionCorp/core)

Scope

`FeeCollector` `NFTRoyaltyFeeCollector` `Treasury` `BaseGauge` `GaugeController` `GaugeRewardsDistributor` `RAACGauge` `RWAGauge` `Governance` `TimelockController` `LLamaTemple` `RAACMinter` `RAACReleaseOrchestrator` `BaseChainlinkFunctionsOracle` `BaseVRFv2Consumer` `RAACHousePriceOracle` `RAACPrimeRateOracle` `ERC20AssetAdapter` `ERC721AssetAdapter` `LendingPool` `LendingPoolStorage` `LiquidationProxy` `VaultProxy` `LiquidationStrategyProxy` `LiquidationSwap` `StabilityPool` `StabilityPoolStorage` `ComplianceRegistry` `RAACHousePrices` `WithCompliance` `DEToken` `DebtToken` `LPToken` `RAACNFT` `RAACToken` `RToken` `RWAIndexToken` `veRAACToken` `ERC20VaultAdapter` `ERC721VaultAdapter` `RWAVault` `BoostCalculator` `LockManager` `PowerCheckpoint` `PercentageMath` `TimeWeightedAverage` `WadRayMath` `DataTypes` `ReserveLibrary` `StringUtils` `Auction` `AuctionFactory` `ZENO` `ZENOFactory`

6. Findings

Findings count

Severity	Amount
Critical	4
High	8
Medium	23
Low	42
Total findings	**77**

Summary of findings

ID	Title	Severity	Status
[C-01]	`_update` inside `DEToken` causes incorrect accounting	Critical	Resolved
[C-02]	Wrong index-based balance calculations	Critical	Resolved
[C-03]	Incorrect `DebtToken` scaling inflates totals miscalculating rates	Critical	Resolved
[C-04]	Incorrect `DEToken` interest causes pool drainage and loss	Critical	Resolved
[H-01]	StabilityPool functions skip the latest LendingPool index update	High	Resolved
[H-02]	`_harvestYield` incorrectly uses total deposits when calculating `totalYield`	High	Resolved
[H-03]	Incorrect utilization rate in deposit and withdraw operations	High	Resolved
[H-04]	LiquidationSwap at risk of sandwich attacks due to spot price reliance	High	Resolved
[H-05]	Bypassing withdrawal timelock via pre-requests and transfers	High	Resolved
[H-06]	Liquidation failure due to incorrect interface signature	High	Resolved

ID	Title	Severity	Status
[H-07]	Incorrect `RToken` scaling inflates supply and breaks reserves	High	Resolved
[H-08]	Access control missing on fee functions enables misuse	High	Resolved
[M-01]	Users can bypass the `LendingPool`'s blacklist when repaying	Medium	Resolved
[M-02]	`RToken` transfer operations use a stale lending pool index	Medium	Resolved
[M-03]	User deposits and admin actions affect `redeemNFT()`	Medium	Resolved
[M-04]	DEToken transfer functions missing access control restrictions	Medium	Resolved
[M-05]	Users unable to fully withdraw token balance from `StabilityPool`	Medium	Resolved
[M-06]	Inconsistent health factor and liquidation threshold causes vulnerability	Medium	Resolved
[M-07]	Borrow cap check bypassed due to stale `totalUsage` value	Medium	Resolved
[M-08]	Liquidity rebalance uses stale `totalLiquidity` in `repay()`	Medium	Resolved
[M-09]	Attacker can block users deposits to Stability Pool	Medium	Resolved
[M-10]	No slippage protection in deposit and NFT redemption functions	Medium	Resolved
[M-11]	Oracle fallback mechanism enables asset undervaluation and unfair liquidations	Medium	Resolved
[M-12]	Incomplete principal withdrawal during vault switching	Medium	Resolved
[M-13]	Swap pair detection does not distinguish liquidity provision from trading	Medium	Acknowledged
[M-14]	Token mismatch in `claimCollectorRewards()` allows discrepancies	Medium	Resolved
[M-15]	Uneven distribution in random NFT selection due to empty adapters	Medium	Resolved

ID	Title	Severity	Status
[M-16]	Insufficient discount protection check	Medium	Resolved
[M-17]	Staleness check missing in NAV oracle usage	Medium	Resolved
[M-18]	Liquidations depend on withdrawals being enabled	Medium	Resolved
[M-19]	Blacklisted accounts bypass restrictions by transferring RToken	Medium	Resolved
[M-20]	Partial yield withdrawals may result in permanent reward loss	Medium	Resolved
[M-21]	Immediate withdrawals are possible by bypassing timelock with token transfer	Medium	Resolved
[M-22]	Oracle update front-running for profit	Medium	Resolved
[M-23]	Supply cap not enforced due to borrow impact and ignored interest	Medium	Resolved
[L-01]	Liquidity value after withdraw in `ensureLiquidity()` is wrong	Low	Resolved
[L-02]	Pausing repay may cause liquidation risk when pool is unpaused	Low	Acknowledged
[L-03]	`amountScaled` calculation in `RToken.mint` is incorrect	Low	Resolved
[L-04]	`Rwavault's` `depositAsset()` and `poolDepositAsset()` return wrong shares	Low	Resolved
[L-05]	Incorrect return value in `getNormalizedDebt()` if time not passed	Low	Resolved
[L-06]	Blacklisted users bypass withdrawal limits if timelock is disabled	Low	Resolved
[L-07]	Missing asset balance validation in vault adapter unregistration	Low	Resolved
[L-08]	No price staleness check in `RAACNFT` minting	Low	Resolved
[L-09]	Collectors allow blacklisted user interaction	Low	Acknowledged
[L-10]	Blacklisted users can withdraw from `StabilityPool`	Low	Resolved
[L-11]	Protocol uses old Curve Liquidity Gauge V5 interface	Low	Resolved

ID	Title	Severity	Status
[L-12]	Actual withdrawal amounts not validated against expected amounts	Low	Resolved
[L-13]	Users can bypass fees by opening vaults and depositing	Low	Resolved
[L-14]	Storage gap absent in `StabilityPoolStorage` risks upgrade collision	Low	Resolved
[L-15]	`DebtToken` decimal precision mismatch with underlying assets	Low	Resolved
[L-16]	Inconsistent same block withdrawal protection implementation	Low	Resolved
[L-17]	Missing validation in `RAACHousePriceOracle`	Low	Resolved
[L-18]	Treasury address can be updated immediately despite defined delay constant	Low	Resolved
[L-19]	Incorrect total supply returned on zero-amount mint	Low	Resolved
[L-20]	Rounding error in percentMul causes fee collection reversion	Low	Resolved
[L-21]	Use `ReentrancyGuardUpgradeable` in `StabilityPool`	Low	Resolved
[L-22]	Immediate liquidation via grace period modification	Low	Acknowledged
[L-23]	Unnecessary approval reset before revert in `distributeFees()`	Low	Resolved
[L-24]	Missing token support validation in `distributeFees()`	Low	Resolved
[L-25]	Minting fee bypass via small deposit rounding	Low	Resolved
[L-26]	Inflation attack in `RWAVault` share calculation via admin actions	Low	Acknowledged
[L-27]	Unsafe NFT transfers without receiver check	Low	Resolved
[L-28]	Redundant role admin assignment in `ComplianceRegistry`	Low	Resolved
[L-29]	Low VRF confirmations increase reorg risk on unstable chains	Low	Acknowledged

ID	Title	Severity	Status
[L-30]	Incorrect contract detection risks fund loss Post-EIP-7702	Low	Acknowledged
[L-31]	ERC20 `approve` misuse may fail fee distribution	Low	**Resolved**
[L-32]	Incorrect operation order in `depositIntoVault()`	Low	**Resolved**
[L-33]	`detoken.totalSupply()` multiplies interest twice	Low	**Resolved**
[L-34]	Frequent small liquidation updates reduce rate gauge	Low	**Resolved**
[L-35]	Missing minimum borrow amount check	Low	**Resolved**
[L-36]	User can bypass `LendingPool`'s `borrowThreshold`	Low	**Resolved**
[L-37]	Without request timestamp validation stale data can overwrite	Low	Acknowledged
[L-38]	Blacklisted users still able to transfer NFTs	Low	**Resolved**
[L-39]	Missing staleness and sequencer checks in Chainlink price feeds	Low	**Resolved**
[L-40]	`batchPriceUpdate()` in `RAACHousePrices` contract is unreachable	Low	Acknowledged
[L-41]	Stability Pool does not account for bad debt after liquidation	Low	Acknowledged
[L-42]	Incompatible oracle interface leads to adapter failures`	Low	**Resolved**

Critical findings

[C-01] `_update` inside `DEToken` causes incorrect accounting

Severity

Impact: High

Likelihood: High

Description

Inside `_update`, which is triggered by `burn`, `mint`, and transfer operations, both `_userDeposits` and `_scaledTotalSupply` are updated.

```
    function _update(address from, address to, uint256 amount) internal override {
        // Setter would change total supply, but here we do not want this to happen so we modify
the user deposits and indexes directly
        // We need to substract out the deposit from the sender
        if(from != address(0) && from != to) {
            _userDeposits[from] -= amount;
        }

        // And add it to the recipient
        if(to != address(0) && to != from) {
>>>         _userDeposits[to] += amount;
            if(_userIndexes[to] == 0) {
                _userIndexes[to] = IStabilityPool(stabilityPool).getPoolIndex();
            } else {
                // We need to calculate the interest earned to be able to adjust the index
                uint256 currentIndex = IStabilityPool(stabilityPool).getPoolIndex();
                uint256 userIndex = _userIndexes[to];
                uint256 userIndexIncrease = currentIndex - userIndex;
                uint256 currentBalance = _userDeposits[to];

                uint256 rawAmountWithInterest = currentBalance +
currentBalance.rayMul(userIndexIncrease);
                uint256 rawInterestAmountToMint = rawAmountWithInterest - currentBalance;

                if(rawInterestAmountToMint > 0) {
                    _userIndexes[to] = currentIndex;
   >>>              _scaledTotalSupply += rawInterestAmountToMint;
                    _mint(to, rawInterestAmountToMint);
                }
            }
        }

        super._update(from, to, amount);
    }
```

```
    function _mint(address account, uint256 value) internal {
        if (account == address(0)) {
            revert ERC20InvalidReceiver(address(0));
        }
        _update(address(0), account, value);
    }
```

```
    function _burn(address account, uint256 value) internal {
        if (account == address(0)) {
            revert ERC20InvalidSender(address(0));
        }
        _update(account, address(0), value);
    }
```

Means when mint is called, since `to != address(0) && to != from` is reached, it will add `amount` to user deposit. Then, when `setUserDeposit` is called inside `StabilityPool` deposit, it will incorrectly update `_scaledTotalSupply`. Additionally, since it recalculates interest accrual, it double-mints interest for the user and also mints interest for the newly deposited tokens.

```
    function _deposit(uint256 rawAmount, uint256 poolIndex) internal {
        // user current index to calculate the scaled amount
        uint256 latestUserIndex = uint256(IRToken(rToken).getUserIndex(msg.sender));

        uint256 previousDeposit = deToken.getUserDeposit(msg.sender);
        uint256 userIndexIncrease = poolIndex - latestUserIndex;

        uint256 scaledAmount;
        if (userIndexIncrease == 0) {
            scaledAmount = rawAmount;
        } else {
            scaledAmount = rawAmount + rawAmount.rayMul(userIndexIncrease);
        }

        // transfer scaled amount from user to this contract
        rToken.safeTransferFrom(msg.sender, address(this), scaledAmount);
        uint256 previousUserIndex = deToken.getUserIndex(msg.sender);
        // @audit - why not use lendingPool latest index?
        uint256 userPreviousIncrease = latestUserIndex - previousUserIndex;
        uint256 currentBalance = deToken.rawBalanceOf(msg.sender);

        uint256 rawAmountWithInterest = currentBalance +
currentBalance.rayMul(userPreviousIncrease);
        uint256 rawInterestAmountToMint = rawAmountWithInterest - currentBalance;
        uint256 newUserDepositAmount = previousDeposit + rawAmount + rawInterestAmountToMint;

        // mint deToken to the user (mint raw amount, balanceOf will return scaled amount)
>>>     deToken.mint(msg.sender, rawAmount + rawInterestAmountToMint);

>>>     deToken.setUserDeposit(msg.sender, newUserDepositAmount);
        deToken.setUserIndex(msg.sender, latestUserIndex);
        depositBlock[msg.sender] = block.number;
```

```
        emit Deposit(msg.sender, scaledAmount, scaledAmount);
    }
```

Since minted amount has already been added to `_userDeposits`, it will not update the `_scaledTotalSupply`.

```
    function setUserDeposit(address user, uint256 amount) external onlyStabilityPool {
        uint256 previousDeposit = _userDeposits[user];
        if(amount == 0) {
            delete _userDeposits[user];
            delete _userIndexes[user];
        } else {
            _userDeposits[user] = amount;
        }
        if(amount > previousDeposit) {
            _scaledTotalSupply += amount - previousDeposit;
        } else {
            _scaledTotalSupply -= previousDeposit - amount;
        }
    }
```

A similar scenario also happens when `burn` is called, since `from != address(0) && from != to` is reached, it will decrease `_userDeposits`. Then, when `setUserDeposit` is called, it will not update the `_scaledTotalSupply`.

Recommendations

Inside `_update`, consider skipping the `mint` and `burn` scenario.

[C-02] Wrong index-based balance calculations

Severity

Impact: High

Likelihood: High

Description

The protocol's index-based accounting system, designed to track interest accrual over time, contains a mathematical error. Instead of calculating compound interest through proportional scaling, the implementation uses a linear approximation that becomes increasingly inaccurate as indexes grow beyond their initial value.

Correct Compound Interest Formula: `Scaled_Balance = Raw_Balance × (Current_Index / User_Last_Index)` Implemented formula: `Scaled_Balance = Raw_Balance + Raw_Balance × (Current_Index - User_Last_Index) / RAY`

This linear approximation is only accurate when User_Last_Index = RAY (1e27).

Consider the scenario:

- User deposits 1,000 tokens when index = 2.0 RAY (protocol has doubled in value).
- Index grows to 2.2 RAY (10% growth period).
- Expected Balance: 1,000 × (2.2 / 2.0) = 1,100 tokens.
- Contract Calculation: 1,000 + 1,000 × (2.2 - 2.0) = 1,200 tokens.

As indexes continue to grow, this error compounds. It leads to wrong RToken, DEToken, and DebtToken interest calculation and balances.

Recommendations

The calculation must be changed from an additive difference based model to a multiplicative ratio based model across all indexed tokens. This ensures that user balances and debts scale correctly in proportion to the index growth.

[C-03] Incorrect `DebtToken` scaling inflates totals miscalculating rates

Severity

Impact: High

Likelihood: High

Description

The `DebtToken` contract incorrectly handles the aggregation of total borrows and applies the interest index multiple times. When users borrow funds through `mint`, the function adds the borrow `amount` directly to `_rawTotalBorrows`. However, this `amount` is already tied to the current `usageIndex`. Later, `totalSupply()` multiplies `_rawTotalBorrows` by the latest `usageIndex`, effectively applying the index twice. This leads to overstating the protocol's total debt.

Similarly, in the `burn` function, repayments reduce `_rawTotalBorrows` by the full repayment `amount`, which also includes accrued interest. As a result, `_rawTotalBorrows` decreases faster than it should, undercutting the actual principal owed.

This causes:

- Inflated or deflated total debt values.
- Incorrect utilization rates, directly impacting calculated borrow and supply interest rates.
- Possible misalignment with borrow caps or protocol risk models.

Example scenario

1. User A borrows `100` when `usageIndex = 1.0`. `_rawTotalBorrows = 100` `totalSupply = 100 * 1.0 = 100`
2. The index grows to `2.0` over time. `totalSupply = 100 * 2.0 = 200`
3. User B borrows another `100` when `usageIndex = 2.0`. `_rawTotalBorrows += 100 = 200` `totalSupply = 200 * 2.0 = 400` However, this means User B's new borrow of `100` is effectively doubled immediately by the current index, overstating the total debt.

Similarly, when repayments occur, the protocol reduces `_rawTotalBorrows` by the full `amount`, including interest, causing it to decrease too aggressively.

Recommendations

Instead of updating `_rawTotalBorrows` with the interest-adjusted `amount`, track scaled principal amounts by dividing by the current `usageIndex`. Then apply the index only in `totalSupply()`.

```
uint256 scaledAmount = amount.rayDiv(usageIndex);
_scaledTotalBorrows += scaledAmount;

function totalSupply() public view returns (uint256) {
    return _scaledTotalBorrows.rayMul(ILendingPool(_lendingPool).getNormalizedDebt());
}
```

When burning (repaying), apply the same scaled reduction:

```
_scaledTotalBorrows -= amount.rayDiv(usageIndex);
```

This ensures the interest index is applied exactly once, maintaining accurate total debt accounting and consistent utilization and interest rates.

[C-04] Incorrect `DEToken` interest causes pool drainage and loss

Severity

Impact: High

Likelihood: High

Description

The `DEToken` contract has multiple flaws in how it calculates interest during token transfers. These issues allow attackers to artificially mint more interest than intended, drain the Stability Pool, and cause users to lose their accumulated interest.

Interest incorrectly calculated on transferred tokens enables Stability Pool drainage

The `_update` function incorrectly calculates interest on the entire `_userDeposits[to]` amount, including newly transferred tokens that should not earn historical interest. This means that when tokens are transferred to an address with `_userIndexes[to] > 0`, the interest calculation is performed on the total deposit after the transfer, rather than only on the previously held balance.

Example scenario:

- Attacker creates multiple fresh addresses (Addr1, Addr2, ..., AddrN).
- Transfers 0 DETokens to each, setting `_userIndexes[addr] = currentPoolIndex`.
- Waits for the pool index to increase.
- Transfers DETokens to Addr1.
- Interest is calculated on the full transferred amount with historical index difference, artificially inflating balance.
- Chains this through Addr1 -> Addr2 -> Addr3, compounding the interest calculation error.
- Withdraws from the Stability Pool, draining real RToken assets.

Recipient receives interest on transferred tokens using an outdated index

The contract uses the recipient's old `_userIndexes` to calculate interest on the new, larger `_userDeposits[to]`, which includes the transferred tokens. This means the recipient effectively earns interest as if they held the new tokens since their earlier index.

For example:

- Pool Index: 1.5.
- Bob has 100 tokens with index 1.1.
- Alice has 1000 tokens with index 1.5.

When Alice transfers 1000 tokens to Bob:

1. Bob's `_userDeposits` becomes 1100.
2. Interest is calculated using Bob's previous index (1.1) and new total deposit (1100).
3. Bob receives interest as if all 1100 tokens had been deposited since index 1.1, earning 1100 × (1.5 - 1.1) = 440, even though Alice's 1000 tokens were just moved.
4. Alice's unclaimed interest on her 1000 tokens is lost entirely.

Sender loses accumulated interest when transferring DEToken

When users transfer DETokens, the interest accumulated from their last `_userIndexes` to the current pool index is not calculated. The `_update` function simply decreases the sender's `_userDeposits`:

```
if(from != address(0) && from != to) {
    _userDeposits[from] -= amount;
}
```

Recommendations

- Update the `_update` function to calculate and mint interest for both the sender (`from`) and the recipient (`to`) **before modifying their** `_userDeposits` **or** `_userIndexes` .
- Ensure that:
- For the sender, compute and mint any accrued interest based on their current `_userDeposits` and `_userIndexes` , then update their index to the current pool index.
- For the recipient, if `_userIndexes[to] == 0` , initialize it to the current pool index; otherwise, calculate and mint interest on their previous balance before adding the new deposit, then update their index.
- Do not include the newly transferred amount when calculating the recipient's interest. Interest should only accrue on balances held prior to the transfer.
- After processing interest and updating indexes for both parties, adjust `_userDeposits` to reflect the transferred amount.
- Ensure `balanceOf` reflects accrued interest to prevent underflows if a user transfers their full visible balance.
- Consider adding explicit tests to simulate transfers between accounts with different indexes to verify that no over-minting or loss of accrued interest occurs.

High findings

[H-01] StabilityPool functions skip the latest LendingPool index update

Severity

Impact: Medium

Likelihood: High

Description

When `deposit` is called, it invokes `ILendingPool(lendingPool).getNormalizedIncome()` to get the `poolIndex` and uses it throughout the operation. However, it does not call `lendingPool.updateState` to update the pool's reserve state data.

```
    function deposit(uint256 scaledAmount) external nonReentrant whenNotPaused
validAmount(scaledAmount) notBlacklisted(msg.sender) {
>>>     uint256 poolIndex = ILendingPool(lendingPool).getNormalizedIncome();
        uint256 userIndex = uint256(IRToken(rToken).getUserIndex(msg.sender));

        // Calculate the user's index increase (how much interest they've earned)
        uint256 userIndexIncrease = poolIndex - userIndex;

        // Convert scaled amount back to raw amount
        uint256 rawAmount;
        if (userIndexIncrease == 0) {
            rawAmount = scaledAmount;
        } else {
            rawAmount = scaledAmount.rayDiv(WadRayMath.RAY + userIndexIncrease);
        }

        uint256 userRTokenBalance = rToken.balanceOf(msg.sender);

        // user has enough balance to cover the scaled amount ?
        if (userRTokenBalance < scaledAmount){
            revert InsufficientBalance();
        }

        _deposit(rawAmount, poolIndex);
    }
```

This causes the operation to use a stale `poolIndex`, resulting in an incorrect amount of tokens and interest being minted. The same issue also exists in the `withdraw` operation.

Recommendations

Trigger `lendingPool.updateState` inside `deposit` and `withdraw` operations.

[H-02] `_harvestYield` incorrectly uses total deposits when calculating `totalYield`

Severity

Impact: Medium

Likelihood: High

Description

When `_harvestYield` is called and `totalYield` is calculated, it uses `totalDeposits`, the total assets deposited into the vault, and multiplies it by the difference between the latest vault price per share and `vaultRewards.lastSharePrice` recorded in the pool.

```
    function _harvestYield() internal {
        uint256 currentSharePrice = _getPricePerShare();

        if (vaultRewards.lastSharePrice == 0 || currentSharePrice <=
vaultRewards.lastSharePrice) {
            // Initialize on first deposit
            if (vaultRewards.lastSharePrice == 0) {
                vaultRewards.lastSharePrice = currentSharePrice;
            }
            return;
        }
        uint256 priceDifference = currentSharePrice - vaultRewards.lastSharePrice;
>>>     uint256 totalYield = (vaultRewards.totalDeposits * priceDifference) / 1e18;

        if (totalYield > 0) {
            uint256 maxWithdrawable = _maxWithdraw(address(this));
            uint256 yieldToWithdraw = totalYield > maxWithdrawable ? maxWithdrawable :
totalYield;
            if (yieldToWithdraw > 0) {
                vaultRewards.unclaimedRewards += yieldToWithdraw;
                _withdraw(yieldToWithdraw, address(this), address(this));
            }
        }

        vaultRewards.lastSharePrice = currentSharePrice;
    }
```

This is incorrect. Consider the case where an `amount` of 1e18 is deposited into the vault and `0.95e18` shares are received.

Later, the share price increases, resulting in a `priceDifference` of `0.1e18` (price per share).

The current calculation for `totalYield` would be: `1e18 * 0.1e18 / 1e18 = 1e17`.

However, the actual yield gained should be based on the number of shares, which is: `0.95e18 * 0.1e18 / 1e18 = 0.95e17`.

This overcalculation of `totalYield` will result in incorrect `unclaimedRewards` and result in lower reserve assets stored in the lending pool.

Recommendations

Use shares when updating `vaultRewards.totalDeposits`.

[H-03] Incorrect utilization rate in deposit and withdraw operations

Severity

Impact: Medium

Likelihood: High

Description

The `updateInterestRatesAndLiquidity` function calculates utilization rates using potentially stale `reserve.totalUsage` values during deposit and withdrawal operations, leading to incorrect interest rate calculations that affect protocol revenue and user returns.

The protocol's utilization rate calculation relies on two key components:

- `reserve.totalLiquidity` - Updated during all operations (deposit, withdraw, borrow, repay, liquidation).
- `reserve.totalUsage` - Only updated during debt operations (borrow, repay, liquidation).

```
    function updateInterestRatesAndLiquidity(ReserveData storage reserve,ReserveRateData
storage rateData,uint256 liquidityAdded,uint256 liquidityTaken) internal {
        ...
        uint256 totalLiquidity = reserve.totalLiquidity;
        uint256 totalDebt = reserve.totalUsage; // @audit totalUsage uses stale value from last
debt operation

        ...

        // Calculate utilization rate
        uint256 utilizationRate = calculateUtilizationRate(reserve.totalLiquidity,
reserve.totalUsage);
        ...

    }
```

We can see that `reserve.totalUsage` is only updated when calling `DebtToken.mint`, `DebtToken.burn`, and `liquidateBorrower`.

```
    function _borrow(address adapter, bytes calldata data, uint256 amount) internal {
        ...
        // Mint DebtTokens to the user (scaled amount)
       (, uint256 amountMinted, uint256 newTotalSupply) =
IDebtToken(reserve.reserveDebtTokenAddress).mint(msg.sender, msg.sender, amount,
reserve.usageIndex, abi.encode(adapter, data));

        // We need to update the position index of the user
        position.positionIndex = reserve.usageIndex;

        // Transfer borrowed amount to user
        IRToken(reserve.reserveRTokenAddress).transferAsset(msg.sender, amount);
        position.rawDebtBalance += amountMinted;
        reserve.totalUsage = newTotalSupply;
        ...
    }
```

```
    function _repay(address adapter, bytes calldata data, uint256 amount, address onBehalfOf)
internal {
        ...
        // Burn DebtTokens from the user whose debt is being repaid (onBehalfOf)
        (, uint256 newTotalSupply, uint256 amountBurned, uint256 balanceIncrease) =
            IDebtToken(reserve.reserveDebtTokenAddress).burn(onBehalfOf, actualRepayAmount,
reserve.usageIndex, abi.encode(adapter, data));

        // We need to update the position index of the user
        position.positionIndex = reserve.usageIndex;

        // Transfer reserve assets from the caller (msg.sender) to the reserve
        IERC20(reserve.reserveAssetAddress).safeTransferFrom(msg.sender,
reserve.reserveRTokenAddress, actualRepayAmount);

        reserve.totalUsage = newTotalSupply;


        ...
    }
```

```
    function finalizeLiquidation(address adapter, address user, bytes calldata data) external
onlyProxy {
        ...
        // Burn DebtTokens from the user
        (uint256 amountScaled, uint256 newTotalSupply, uint256 amountBurned, uint256
balanceIncrease) =
            IDebtToken(reserve.reserveDebtTokenAddress).burn(user, positionDebt,
reserve.usageIndex, abi.encode(adapter, data));
        ...
        reserve.totalUsage = newTotalSupply;

        // Update liquidity and interest rates
        ReserveLibrary.updateInterestRatesAndLiquidity(reserve, rateData, amountScaled, 0);
        emit LiquidationFinalized(stabilityPool, user, adapter, data, positionDebt,
IAssetAdapter(adapter).getAssetValue(user, data));
    }
```

When debt has accrued interest but `totalUsage` hasn't been updated, actual utilization is higher than calculated. Lower calculated utilization leads to suppressed borrowing and supply rates. It leads to less protocol fee collection and wrong borrowing fees.

Recommendations

Update the `reserve.totalUsage` before calculating `utilizationRate`.

[H-04] LiquidationSwap at risk of sandwich attacks due to spot price reliance

Severity

Impact: Medium

Likelihood: High

Description

The LiquidationSwap contract is vulnerable to sandwich attacks because it relies on the spot price returned by `liquidityPool.get_dy` for slippage protection calculations. This creates an issue where an attacker can manipulate the pool's spot price before the liquidation transaction, making the slippage protection mechanism ineffective.

```
    function _swap(address sender, uint256 amount) internal {
        ...
        uint256 expectedDy = liquidityPool.get_dy(1, 0, amount);
        if (expectedDy == 0) revert InvalidExchange();
        uint256 minDy = enableSlippageProtection ? (expectedDy * (10000 -
slippageProtectionBps)) / 10000 : 0;
        // Exchange index token (1) for crvUSD (0) since we swapped the order in the pool
        liquidityPool.exchange(1, 0, amount, minDy);
        ...
    }
```

RAAC-NFT liquidations representing real estate assets can involve substantial amounts, and the default `swapPercentage` is typically 90% to cover debt repayment. Therefore, the liquidation can be the target because of its high value.

It leads to value loss for the Stability Pool.

Recommendations

The slippage protection percentage should be applied to a weighted average price instead of spot price or consider other approach (need to avoid blocking liquidation).

[H-05] Bypassing withdrawal timelock via pre-requests and transfers

Severity

Impact: Medium

Likelihood: High

Description

A design flaw in the `requestWithdraw` function allows users to completely bypass the protocol's withdrawal timelock and same-block flash-loan protection mechanisms. The vulnerability stems from the lack of balance validation when queuing withdrawal requests, which can be exploited through a simple token transfer strategy to enable instant withdrawals of newly deposited funds.

The issue lies in the requestWithdraw function's failure to validate that the requesting user actually possesses the RTokens they wish to withdraw:

```
    function requestWithdraw(uint256 amount) external whenNotPaused()
notBlacklisted(msg.sender) {
        ...
        timelock.amount = amount; // @audit no balance check
        timelock.readyAt = uint64(block.timestamp + parameters.withdrawTimelockDuration);
        timelock.expireAt = uint64(block.timestamp + parameters.withdrawTimelockDuration +
parameters.withdrawTimelockDelay);

        emit WithdrawQueued(msg.sender, amount, timelock.readyAt);
    }
```

This allows users to queue withdrawal requests for arbitrary amounts without ownership validation, effectively pre-authorizing future withdrawals before acquiring the necessary tokens.

Consider a scenario:

- An attacker can use two addresses (or a collaborator) to execute this exploit. Let's call them Bob (Attacker) and Alice (Depositor).
- Bob Queues Withdrawal: Bob, who has an RToken balance of zero, calls `requestWithdraw(1000)`. Since there is no balance validation, the transaction succeeds, and a withdrawal request is queued for his address.
- Bob waits for the withdrawal timelock period (e.g., 30 minutes) to expire. His withdrawal request is now "ready" to be executed.
- In a new block, Alice deposits 1000 crvUSD into the LendingPool and receives 1000 RToken.
- Alice immediately transfers her 1000 RToken to Bob. This is a standard ERC20 transfer and is not tracked by the LendingPool's internal state.
- Bob immediately calls `withdraw(1000)`. The LendingPool performs the following checks against Bob's address (msg.sender):
  - The `CannotWithdrawInSameBlock` check (`depositBlock[Bob] == block.timestamp`) passes because `depositBlock[Bob]` is 0. The flash-loan protection is bypassed.
  - The timelock check (`_executeWithdrawTimelock`) passes because Bob's withdrawal request is valid and the timelock has expired. The withdrawal timelock is bypassed.
- Alice's funds, which should have been subject to a withdrawal delay, have been withdrawn instantly.

Recommendations

Add a balance check in `requestWithdraw()` to ensure users can only request withdrawals for funds they actually possess:

```
function requestWithdraw(uint256 amount) external whenNotPaused() notBlacklisted(msg.sender) {
    ...

    // Add balance validation
    require(RToken(reserve.reserveRTokenAddress).balanceOf(msg.sender) >= amount,
            "Insufficient balance for withdrawal request");
    }

    ...
}
```

[H-06] Liquidation failure due to incorrect interface signature

Severity

Impact: Medium

Likelihood: High

Description

The `LiquidationSwap` contract uses an incorrect interface when interacting with Curve's Twocrypto-NG pools, causing all liquidation attempts to revert. This vulnerability completely disables the protocol's liquidation mechanism, potentially leading to the accumulation of bad debt and putting the stability pool at risk.

```
    function _swap(address sender, uint256 amount) internal {
        ...
        uint256 expectedDy = liquidityPool.get_dy(1, 0,
amount); // @audit This call will revert due to signature mismatch
        if (expectedDy == 0) revert InvalidExchange();
        uint256 minDy = enableSlippageProtection ? (expectedDy * (10000 -
slippageProtectionBps)) / 10000 : 0;
        liquidityPool.exchange(1, 0, amount,
minDy); // @audit This call will also revert due to signature mismatch
        ...
    }
```

The `LiquidationSwap` contract implements an interface (ILiquidityPool) that expects `int128` parameters for the `exchange` and `get_dy` functions:

```
interface ILiquidityPool {
    // Core Curve-like functions
    function exchange(int128 i, int128 j, uint256 dx, uint256 min_dy) external returns
(uint256);
    function get_dy(int128 i, int128 j, uint256 dx) external view returns (uint256);
    ...
}
```

However, the actual Curve Twocrypto-NG pool implementation uses `uint256` parameters for these same functions:

```
@external
@nonreentrant
def exchange(
    i: uint256,
    j: uint256,
    dx: uint256,
    min_dy: uint256,
    receiver: address = msg.sender
) -> uint256:
    """
    @notice Exchange using wrapped native token by default
    @param i Index value for the input coin
    @param j Index value for the output coin
    @param dx Amount of input coin being swapped in
    @param min_dy Minimum amount of output coin to receive
    @param receiver Address to send the output coin to. Default is msg.sender
    @return uint256 Amount of tokens at index j received by the `receiver
    """
    ...

def get_dy(i: uint256, j: uint256, dx: uint256) -> uint256:
```

```
    """
    @notice Get amount of coin[j] tokens received for swapping in dx amount of coin[i]
    @dev Includes fee.
    @param i index of input token. Check pool.coins(i) to get coin address at ith index
    @param j index of output token
    @param dx amount of input coin[i] tokens
    @return uint256 Exact amount of output j tokens for dx amount of i input tokens.
    """
    view_contract: ITwocryptoView = params._views_implementation()
    return staticcall view_contract.get_dy(i, j, dx, self)
```

https://github.com/curvefi/twocrypto-ng/blob/main/contracts/main/Twocrypto.vy

The result is that all calls to `liquidateBorrower` will revert. It leads to bad debt accumulation, and the stability pool cannot recover funds from liquidations, potentially becoming insolvent.

Recommendations

Using the correct interface when interacting with Twocrypto-NG pool.

[H-07] Incorrect `RToken` scaling inflates supply and breaks reserves

Severity

Impact: Medium

Likelihood: High

Description

The `RToken` contract incorrectly manages the aggregation of total deposits and interest index scaling. During `mint`, it directly adds the user's deposit `amount` to `_rawTotalDeposits`. However, this `amount` is already adjusted by the current `liquidityIndex`, reflecting interest accrued up to that point.

When `totalSupply()` later multiplies `_rawTotalDeposits` by the current `liquidityIndex`, it applies the interest index a second time. This results in the total supply being overstated, disrupting accurate reserve and utilization calculations. It also breaks functions like `calculateDustAmount()`, causing it to return zero or incorrect values even when excess funds (dust) exist.

Similarly, in the `burn` function, the contract reduces `_rawTotalDeposits` by the full withdrawal `amount`, which includes accumulated interest. This causes `_rawTotalDeposits` to decrease faster than intended, underreporting the actual deposited principal.

Example scenario

1. User deposits `100` when `liquidityIndex = 1.0`. `_rawTotalDeposits = 100` `totalSupply = 100 * 1.0 = 100`
2. The index grows to `1.05` from accrued yield. `totalSupply = 100 * 1.05 = 105`
3. Another user deposits `50` when `liquidityIndex = 1.05`. `_rawTotalDeposits += 50 = 150` `totalSupply = 150 * 1.05 = 157.5` However, the true combined supply should be: `(100 * 1.05) + (50 * 1.05 / 1.05) = 105 + 50 = 155`

This demonstrates how the protocol overstates the actual total deposits due to applying the interest index twice.

Recommendations

Instead of adding amounts that already include the current `liquidityIndex`, track scaled principal balances by dividing by the index at the time of the mint or burn operation. Then apply the index only once inside `totalSupply()`.

```
uint256 scaledAmount = amount.rayDiv(liquidityIndex);
_scaledTotalDeposits += scaledAmount;

function totalSupply() public view returns (uint256) {
    return _scaledTotalDeposits.rayMul(ILendingPool(_lendingPool).getNormalizedIncome());
}
```

When burning (withdrawing), apply the same inverse scaling to ensure symmetry:

```
_scaledTotalDeposits -= amount.rayDiv(liquidityIndex);
```

This approach ensures that interest is applied exactly once, maintaining accurate total supply reporting, utilization metrics, and reserve calculations.

[H-08] Access control missing on fee functions enables misuse

Severity

Impact: High

Likelihood: Medium

Description

The `FeeCollector` contract exposes critical functions — namely `claimCollectorRewards` and `claimNFTUnderlying` — which are callable by anyone without any form of access control or role restrictions. This creates severe opportunities for malicious actors to disrupt the protocol's revenue accounting and fee distribution logic by front-running or deliberately misusing these functions.

Unrestricted `claimCollectorRewards`

The `claimCollectorRewards` function is intended to pull rewards from whitelisted `target` contracts (like LendingPools or Vaults) and allocate them according to the specified `feeType`. However, it is fully permissionless:

```
function claimCollectorRewards(address token, address target, bytes32 feeType)
    external nonReentrant whenNotPaused returns (bool) {
```

This means **any external user can invoke reward claims at any time**, specifying arbitrary valid `feeType` values. Since different `feeTypes` dictate distinct distribution percentages (e.g., 50% Treasury / 50% veRAAC for `VAULT_YIELD_FEE` vs. 90% Treasury / 10% Burn for `NFT_ROYALTY_FEE`), an attacker can front-run legitimate reward distributions to force allocations under less favorable splits.

Example scenario:

- The protocol intends to claim and distribute `VAULT_YIELD_FEE` rewards (50/50 split).
- An attacker front-runs this by calling `claimCollectorRewards` with the same target and token, but sets `feeType` to `NFT_ROYALTY_FEE`.
- Funds get allocated with a different distribution ratio, undermining the intended protocol economics.

This was originally highlighted in findings [H-11] and [M-32].

Unrestricted `claimNFTUnderlying`

Similarly, the `claimNFTUnderlying` function, used to extract crvUSD (or other tokens) paid for NFT minting, also lacks any access control:

```
function claimNFTUnderlying(address token, address target, uint256 amount, bytes32 feeType)
    external nonReentrant whenNotPaused returns (bool) {
```

This allows any actor to trigger withdrawals from whitelisted NFT targets, specifying any `feeType` to misallocate collected fees.

Example:

- `RAAC_CORP` has 100,000 crvUSD of minting fees meant for its revenue pool.
- An attacker front-runs with `claimNFTUnderlying`, setting the `feeType` to `ROYALTY_FEE` (which distributes to veRAAC holders).
- The funds are now accounted under the wrong `feeType`, resulting in unintended beneficiaries receiving the funds.

Impact:

- **Direct manipulation of protocol fee flows**, bypassing governance or operator-controlled scheduling of fee distributions.
- **Permanent misallocation of revenue**, undermining investor and treasury interests.
- **Potential front-running attacks**, exploiting predictable distribution calls queued by protocol operators.

Recommendation

Implement robust access control on critical functions:

- Restrict `claimCollectorRewards` and `claimNFTUnderlying` to be callable only by an authorized admin or operator role (e.g., using `onlyOwner` or RBAC pattern).
- This ensures that fee claiming and distribution can only be triggered by trusted actors under the protocol's controlled schedule, preventing manipulation of `feeType` allocations and front-running of legitimate distributions.

Medium findings

[M-01] Users can bypass the `LendingPool`'s blacklist when repaying

Severity

Impact: Medium

Likelihood: Medium

Description

When `repayOnBehalf` operation is performed, it only checks if the sender is not blacklisted.

```
    function repayOnBehalf(address adapter, bytes calldata data, uint256 amount, address
onBehalfOf) external nonReentrant whenNotPaused onlyValidAmount(amount)
onlySupportedAdapter(adapter) notBlacklisted(msg.sender) {
        if (!parameters.canPaybackDebt) revert PaybackDebtDisabled();
        if (onBehalfOf == address(0)) revert AddressCannotBeZero();

        // Check if the repayment meets the minimum threshold (1% of the scaled debt)
        // This prevents malicious users from making tiny repayments to force interest
capitalization
        uint256 positionScaledDebt = getPositionScaledDebt(adapter, onBehalfOf, data);
        if (positionScaledDebt != 0 && msg.sender != onBehalfOf) {
            // PERCENTAGE_FACTOR = 1e4; = 10000
            uint256 minRepaymentAmount = positionScaledDebt.percentMul(100); // 1% = 100 in
PercentageMath (basis points)
            if (amount < minRepaymentAmount) {
                revert RepaymentBelowMinimumThreshold();
            }
        }

        _repay(adapter, data, amount, onBehalfOf);
    }
```

However, `onBehalfOf` is not checked, allowing a blacklisted address to repay by using another address as the caller of `repayOnBehalf`.

Recommendations

Also, check whether `onBehalfOf` is blacklisted.

[M-02] `RToken` transfer operations use a stale lending pool index

Severity

Impact: low

Likelihood: High

Description

When `RToken` `_update` is triggered, it will use `ILendingPool(_lendingPool).getNormalizedIncome` to update the users balance interest.

```
    function _update(address from, address to, uint256 amount) internal override {
>>>     uint256 currentLiquidityIndex = ILendingPool(_lendingPool).getNormalizedIncome();
        if (from != address(0)) {
            // Update user index for sender to current liquidity index to prevent interest
exploitation
            uint256 senderBalance = super.balanceOf(from);
            uint256 senderIncrease = senderBalance.rayMul(currentLiquidityIndex) -
senderBalance.rayMul(_userState[from].index);
            _userState[from].index = currentLiquidityIndex.toUint128();
            if(senderIncrease > 0){
                _mint(from, senderIncrease.toUint128());
            }
        }

        // Only update recipient if different from sender
        if (to != address(0) && to != from) {
            // Update user index for recipient to current liquidity index to prevent interest
exploitation
            uint256 recipientBalance = super.balanceOf(to);
            uint256 recipientIncrease = recipientBalance.rayMul(currentLiquidityIndex) -
recipientBalance.rayMul(_userState[to].index);
            _userState[to].index = currentLiquidityIndex.toUint128();
            if(recipientIncrease > 0){
                _mint(to, recipientIncrease.toUint128());
            }
        }

        super._update(from, to, amount);
    }
```

However, since `lendingPool.updateState` is not called beforehand, it may not use the latest `reserve.liquidityIndex`.

Recommendations

Consider to trigger `lendingPool.updateState` inside `_update`.

[M-03] User deposits and admin actions affect `redeemNFT()`

Severity

Impact: Medium

Likelihood: Medium

Description

When users call `redeemNFT`, it invokes `getNextRandomNFT` to determine the `adapter` and `tokenId` to be received, using Chainlink VRF for randomness.

```
function getNextRandomNFT() public view returns (address adapter, uint256 tokenId) {
    if (address(vrfConsumer) == address(0)) revert InvalidVRFAddress();
    IBaseVRFv2Consumer.RequestStatus memory requestStatus = vrfConsumer.getRequestStatus();
    if (!requestStatus.fulfilled) revert IBaseVRFv2Consumer.RequestNotFulfilled();

    uint256 aCount = redeemableERC721Adapters.length;
    require(aCount > 0, "RWAVault: no redeemable adapters");
    uint256 rand = requestStatus.randomWords[0];
    uint256 start = rand % aCount;
    uint256 i = start;
    // circular-scan until we find a non-empty adapter
    do {
        address candidate = redeemableERC721Adapters[i];

        uint256 balance = IERC721VaultAdapter(candidate).getDepositedTokensCount();

        if (balance > 0) {
            adapter = candidate;
            // select a token index pseudo-randomly
            uint256 idx = uint256(keccak256(abi.encode(rand, i))) % balance;
            tokenId = IERC721VaultAdapter(candidate).getDepositedTokenAtIndex(idx);
            return (adapter, tokenId);
        }

        i = (i + 1) % aCount;
    } while (i != start);
    revert("RWAVault: no NFTs available");
}
```

When a user calls this, the actual token received will be random, but it's based on the existing adapters and `tokenIds` that were previously known to the user. However, other users may deposit into the vault, or the admin may deposit or withdraw NFTs, potentially affecting the output of `getNextRandomNFT` and causing the user to receive an unexpected token.

Recommendations

Consider to adjust the random selection mechanism so that other operations doesn't heavily impact the result.

[M-04] DEToken transfer functions missing access control restrictions

Severity

Impact: Low

Likelihood: High

Description

According to the protocol, DE token can only be transferred by Stability Pool Only. However, anyone can transfer the DEToken. The issue stems from incomplete access control implementation. The transfer functions were overridden without applying the `onlyStabilityPool` modifier.

```
    function transfer(address recipient, uint256 amount) public override(ERC20,IERC20) returns
(bool) {
        return super.transfer(recipient, amount);
    }


    function transferFrom(address sender, address recipient, uint256 amount) public
override(ERC20,IERC20) returns (bool) {
        return super.transferFrom(sender, recipient, amount);
    }
```

Recommendations

Add `onlyStabilityPool` to `transfer` and `transferFrom` functions.

[M-05] Users unable to fully withdraw token balance from `StabilityPool`

Severity

Impact: Low

Likelihood: High

Description

The StabilityPool contract contains a design error that prevents users from withdrawing their complete token balance due to a mismatch between the withdrawal request mechanism and the timelock validation logic.

- When users call `requestWithdraw`, the system validates that the requested amount does not exceed their current deToken balance.
- When users call `withdraw` after the timelock period, the system requires the withdrawal amount to exactly match the originally requested amount.
- However, during the 30-minute timelock period, the user's balance grows due to accrued interest.

```
    function requestWithdraw(uint256 amount) external nonReentrant whenNotPaused
notBlacklisted(msg.sender) {
        ...
        if (amount > deToken.balanceOf(msg.sender)) revert InsufficientBalance();
        WithdrawTimelock storage timelock = withdrawTimelock[msg.sender];
        timelock.amount = amount;
        ...
    }

    function _withdrawTimelockCheck(address user, uint256 amount) internal {
        ...
        if (userTimelock.amount != amount) revert WithdrawTimelockInvalid();

        ...
    }
```

Recommendations

It's recommended to allow users to withdraw all balance by setting the amount to `type(uint256).max`.

[M-06] Inconsistent health factor and liquidation threshold causes vulnerability

Severity

Impact: High

Likelihood: Low

Description

The protocol uses inconsistent comparison operators when checking the liquidation threshold:

- Liquidation Logic (initiateLiquidation): Allows liquidation when `healthFactor ≤ HEALTH_FACTOR_LIQUIDATION_THRESHOLD`

```
    function initiateLiquidation(address adapter, address user, bytes calldata data) external
onlyProxy {
        ...
        if (healthFactor > HEALTH_FACTOR_LIQUIDATION_THRESHOLD) revert HealthFactorTooHigh();

        ...
    }
```

- Withdrawal Logic (withdrawAsset): Prevents withdrawal when `newHealthFactor < HEALTH_FACTOR_LIQUIDATION_THRESHOLD`

```
    function withdrawAsset(address adapter, bytes calldata data) external nonReentrant
whenNotPaused onlySupportedAdapter(adapter) notBlacklisted(msg.sender) {
        ...
            if (newHealthFactor < HEALTH_FACTOR_LIQUIDATION_THRESHOLD) {
                revert WithdrawalWouldLeaveUserUnderCollateralized();
            }
        ...
    }
```

This creates a problematic edge case where: - Users can withdraw assets until their health factor reaches exactly 1.0. - Users can be liquidated when their health factor is exactly 1.0.

The documentation states: "Marks position under liquidation when health factor < 1", indicating liquidation should only occur when health factor is strictly less than 1.0, not equal to 1.0.

It leads to users maintaining exactly 1.0 health factor, which can be immediately liquidated despite having a "healthy" position according to withdrawal logic.

Recommendations

Modify the initiateLiquidation to:

```
    function initiateLiquidation(address adapter, address user, bytes calldata data) external
onlyProxy {
        ...
-        if (healthFactor > HEALTH_FACTOR_LIQUIDATION_THRESHOLD) revert HealthFactorTooHigh();
+        if (healthFactor >= HEALTH_FACTOR_LIQUIDATION_THRESHOLD) revert HealthFactorTooHigh();

        ...
    }
```

[M-07] Borrow cap check bypassed due to stale `totalUsage` value

Severity

Impact: Low

Likelihood: High

Description

The `_validateBorrow` function in the lending pool incorrectly validates borrow amounts against the configured borrow cap by using a stale `reserve.totalUsage` value that does not account for accrued interest since the last state update. This allows borrowers to exceed the intended borrow cap when interest has accumulated but has not been realized through recent borrow/repay operations.

```
    function _validateBorrow(address adapter, bytes calldata data, uint256 amount) internal
view {
        ...
        if (reserve.totalUsage + amount > parameters.borrowCap) revert BorrowCapReached();
        ...
    }
```

The `reserve.totalUsage` value represents the last recorded total borrowed amount but does not include interest that has accrued since the last state-changing operation (borrow/repay/liquidation). This creates a discrepancy between the actual outstanding debt and the value used for cap validation.

It undermines the borrow cap mechanism designed to limit exposure.

Recommendations

It is recommended to modify _validateBorrow to calculate the current total debt, including accrued interest, before comparing it against the borrow cap.

[M-08] Liquidity rebalance uses stale `totalLiquidity` in `repay()`

Severity

Impact: Low

Likelihood: High

Description

The `_repay` function contains a logical issue in the order of operations that causes liquidity rebalancing to be performed using outdated `totalLiquidity` values. Specifically, `_rebalanceLiquidity` is called before `updateInterestRatesAndLiquidity`, which means the rebalancing calculation relies on stale liquidity data that doesn't account for the current repayment transaction. It leads to suboptimal liquidity allocation.

```
    function _repay(address adapter, bytes calldata data, uint256 amount, address onBehalfOf)
internal {
        ...

        _rebalanceLiquidity();

        // Update liquidity and interest rates
        ReserveLibrary.updateInterestRatesAndLiquidity(reserve, rateData, actualRepayAmount, 0);

        emit Repay(msg.sender, onBehalfOf, actualRepayAmount);
    }
```

Recommendations

Update the liquidity state before performing rebalancing.

[M-09] Attacker can block users deposits to Stability Pool

Severity

Impact: Medium

Likelihood: Medium

Description

A malicious actor can prevent any user from depositing RToken into the Stability Pool by simply transferring 1 wei of DEToken to them. This attack exploits an inconsistency in how user indexes are managed between DEToken transfers and Stability Pool deposits, causing an arithmetic underflow that reverts the deposit transaction.

The vulnerability stems from a mismatch in user index management between the two contracts: - DEToken: When users receive DEToken via transfer, their `_userIndexes` is automatically set to the current pool index. - StabilityPool: When calculating interest during deposits, it assumes the DEToken user index is always ≤ the RToken user index.

`DEToken._update` function:

```
    function _update(address from, address to, uint256 amount) internal override {
        ...
        if(to != address(0) && to != from) {
```

```
            _userDeposits[to] += amount;
            if(_userIndexes[to] == 0) {
                _userIndexes[to] = IStabilityPool(stabilityPool).getPoolIndex(); // @audit Sets
user index to current pool index
            } else {
                // We need to calculate the interest earned to be able to adjust the index
                uint256 currentIndex = IStabilityPool(stabilityPool).getPoolIndex();
                ...

                if(rawInterestAmountToMint > 0) {
                    _userIndexes[to] = currentIndex; // @audit Sets user index to current pool
index
                    _scaledTotalSupply += rawInterestAmountToMint;
                    _mint(to, rawInterestAmountToMint);
                }
            }
        }
        ...
    }
```

`StabilityPool._deposit` function:

```
    function _deposit(uint256 rawAmount, uint256 poolIndex) internal {
        // user current index to calculate the scaled amount
        uint256 latestUserIndex = uint256(IRToken(rToken).getUserIndex(msg.sender));
        ...
        uint256 previousUserIndex = deToken.getUserIndex(msg.sender);
        uint256 userPreviousIncrease = latestUserIndex - previousUserIndex; // @audit underflow
because latestUserIndex < previousUserIndex
        ...

        emit Deposit(msg.sender, scaledAmount, scaledAmount);
    }
```

Consider a scenario:

- Initial state: Alice wants to deposit RToken into the Stability Pool.
  - Alice's RToken user index: 1.05e27 (slightly behind current pool index).
  - Current pool index: 1.06e27.
  - Alice's DEToken user index: 0 (never received DEToken before).
- Attack: Bob (malicious actor) transfers 1 wei of DEToken to Alice.
  - This triggers `DEToken._update`.
  - Alice's DEToken user index is set to: 1.06e27 (current pool index).
- Alice attempts to deposit RToken.
  - latestUserIndex = 1.05e27 (from RToken).
  - previousUserIndex = 1.06e27 (from DEToken, updated by Bob's transfer).
  - Calculation: 1.05e27 - 1.06e27 = UNDERFLOW.
  - Transaction reverts with arithmetic error.

Legitimate users are blocked from depositing RToken to Stability Pool.

Recommendations

Refactor the `Stability._deposit` function to handle the case.

[M-10] No slippage protection in deposit and NFT redemption functions

Severity

Impact: Medium

Likelihood: Medium

Description

The RWAVault contract lacks slippage protection mechanisms in `depositAsset` and `redeemNFT`.

Deposit operations without minimum share guarantees: When users call depositAsset, they cannot specify the minimum number of shares they expect to receive. The share calculation depends on the current `totalAssets` and `totalSupply`, which can change between transaction submission and execution due to asset value fluctuations and fee adjustment.

```
    function depositAsset(address adapter, bytes calldata data, address receiver) external
override onlySupportedDepositableAdapter(adapter) notBlacklisted(msg.sender) nonReentrant
returns (uint256 sharesMinted) {
        // Only apply discount when msg.sender is the same as receiver to prevent proxy
exploitation
        uint256 discountPercentage = 0;
        if (msg.sender == receiver) {
            discountPercentage = _calculateLlamaPerkDiscount(msg.sender);
        }
        uint256 mintingFee = (indexTokenMintingFee * (100_00 - discountPercentage)) / 100_00;
        return _deposit(adapter, data, receiver, mintingFee);
    }
```

NFT redemption without maximum share limit: In `redeemNFT`, users cannot specify the maximum number of shares they're willing to burn for a randomly selected NFT. The function calculates `sharesBurned = convertToShares(nftPrice)` without allowing users to set an upper bound, potentially forcing them to burn more shares than expected if the NFT's value or vault share price changes unfavorably.

```
    function redeemNFT() external override nonReentrant notBlacklisted(msg.sender) {
        if (address(vrfConsumer) == address(0)) revert InvalidVRFAddress();

        (address adapter, uint256 tokenId) = getNextRandomNFT();

        bytes memory data = abi.encode(tokenId);
```

```
        uint256 nftPrice = IVaultAssetAdapter(adapter).getAssetValue(data);
        if (nftPrice == 0) revert InvalidNFT();
        uint256 sharesBurned = convertToShares(nftPrice);
        if (IVaultToken(vaultToken).balanceOf(msg.sender) < sharesBurned) revert
InsufficientBalance();

        IVaultToken(vaultToken).burn(msg.sender, sharesBurned);
        IVaultAssetAdapter(adapter).withdraw(data, msg.sender);
        // send the request for the next token to redeem
        vrfConsumer.requestRandomWords();

        emit RedeemNFT(msg.sender, IVaultAssetAdapter(adapter).getAssetToken(), tokenId,
sharesBurned);
    }
```

Recommendations

It's recommended to add slippage protection to `depositAsset` and `redeemNFT`.

[M-11] Oracle fallback mechanism enables asset undervaluation and unfair liquidations

Severity

Impact: High

Likelihood: Low

Description

The `RAACHousePrices` oracle contains an issue in its circuit breaker fallback mechanism. When crvUSD de-pegs below a threshold (e.g., $0.90) and `circuitBreakerEnabled` is enabled, the oracle incorrectly assumes a `1:1` USD instead of using the actual exchange rate.

```
    function getLatestPrice(uint256 _tokenId) external view returns (uint256 _crvUSDPrice,
uint256 _lastUpdateTimestamp) {
        uint256 usdPrice = tokenToHousePrice[_tokenId];
        ...
            uint8 decimals = dataFeed.decimals();
            (, int256 answer, , , ) = dataFeed.latestRoundData();

            if (answer <= 0) return _fallBackHousePrice(_tokenId);

            if (circuitBreakerEnabled) {
                uint256 adjustedMinThreshold = _adjustThresholdToDecimals(minPriceThreshold,
decimals);
                uint256 adjustedMaxThreshold = _adjustThresholdToDecimals(maxPriceThreshold,
decimals);

                if (uint256(answer) < adjustedMinThreshold || uint256(answer) >
```

```
adjustedMaxThreshold) {
                    return _fallBackHousePrice(_tokenId);
                }
            }
            _crvUSDPrice = (usdPrice * 10 ** decimals) / uint256(answer);
        }
        _lastUpdateTimestamp = tokenToLastUpdateTimestamp[_tokenId];
    }

    function _fallBackHousePrice(uint256 _tokenId) internal view returns (uint256 _crvUSDPrice,
uint256 _lastUpdateTimestamp) {
        _crvUSDPrice = tokenToHousePrice[_tokenId]; // @audit Returns USD price directly,
assuming 1 crvUSD = 1 USD
        _lastUpdateTimestamp = tokenToLastUpdateTimestamp[_tokenId];
    }
```

This creates two severe vulnerabilities:

1. Discounted NFT redemptions from the RWAVault (up to 20% discount)

   Instead of using the correct crvUSD price provided by Chainlink, `_fallBackHousePrice` returns the NFT's base price in USD, effectively assuming that 1 `crvUSD` is equal to 1 `USD`. This leads to an undervaluation of assets during a `crvUSD` de-peg event, which can be exploited to drain value from the `RWAVault`.

   Consider a scenario:

- Initial State: An `RAACNFT` in the `RWAVault` is worth $100,000. The `crvUSD` price is stable at $1.00. The vault correctly values the NFT at 100,000 `crvUSD`.
- De-peg Event: The price of `crvUSD` drops to $0.80. The true value of the $100,000 NFT is now `$100,000 / $0.80` = 125,000 `crvUSD`.
- Circuit Breaker trigger: The `RAACHousePrices` contract detects that the `crvUSD` price ($0.80) is below its minimum threshold (e.g., $0.90) and triggers the fallback mechanism.
- Incorrect Valuation: The oracle now reports the NFT's value as 100,000 `crvUSD`, incorrectly assuming a 1:1 peg with USD. The asset is now undervalued by 20%.
- The Attack: An attacker calls the `RWAVault.redeemNFT` function. The vault uses the flawed oracle price and calculates that the attacker only needs to burn `iRAAC` shares worth 100,000 `crvUSD` to redeem the NFT.
- Value Extraction: The attacker successfully redeems the NFT, acquiring an asset worth 125,000 `crvUSD` at a cost of only 100,000 `crvUSD` in vault shares.

   The victims are all other `iRAAC` token holders, whose shares are now backed by fewer assets, leading to a permanent dilution of the vault's Net Asset Value (NAV).

1. Cliff-edge liquidations due to discontinuous pricing

Consider a scenario:

- Initial State: A user has a position with collateral valued at 100,000 `crvUSD` and a Health Factor of 1.3. The position is safe.
- Moderate De-peg: `crvUSD` de-pegs to $0.91. The standard pricing formula now reports the collateral value as `100,000 / 0.91` ≈ 109,890 `crvUSD`. The user's Health Factor increases to ~1.43. The system incorrectly signals that the position is safer.
- Increased borrowing: the user borrows more `crvUSD`, bringing their Health Factor down to what they believe is a still-safe level of 1.2.
- The Cliff: The price of `crvUSD` moves slightly from $0.91 to $0.89 This small change trips the circuit breaker's $0.90 threshold.
- Repricing: The pricing logic instantly switches to the fallback, and the user's collateral value plummets from 109,890 `crvUSD` back down to 100,000 `crvUSD`.
- Unfair Liquidation: The user's Health Factor is recalculated with the new, lower collateral value: `(100,000 * 0.8) / NewDebt` = 0.99. The user's position is now under 1.0 and is immediately liquidatable.

The user was liquidated because of non-continuous pricing mechanism in the protocol itself.

Recommendations

It's recommended to pause the protocol if the circuit breaker triggers or using the correct price returns from oracle.

[M-12] Incomplete principal withdrawal during vault switching

Severity

Impact: Medium

Likelihood: Medium

Description

The `setVault()` function in LendingPool.sol attempts to withdraw all deposits from the old vault before switching, but withdraws `vaultRewards.totalDeposits` amount. There are 2 issues: 1. This could revert, especially if the vault has suffered losses or if there are slippage issues during withdrawal. 2. When the withdrawal fails, the function continues without reverting, leaving the user's funds still trapped.

```
        // If we change the vault, we need to claim all rewards from the old vault
        if (address(vault) != address(newVault) && address(vault) != address(0)) {
            (bool success,) = vaultProxy.delegatecall(
                abi.encodeWithSignature("withdrawFromVault(uint256)",
vaultRewards.totalDeposits)
            );
```

```
            if (!success) {
                // We do not revert because deposit/withdraw always take entire yield out.
                emit VaultWithdrawalFailed(address(vault));
            }
        }
```

The function continues with vault switching even if the withdrawal fails, potentially leaving user funds trapped in the old vault.

Recommendations

Calculate the actual shares to withdraw based on the current vault state and revert if the withdrawal fails.

[M-13] Swap pair detection does not distinguish liquidity provision from trading

Severity

Impact: Medium

Likelihood: Medium

Description

In the `_swapAwareTransfer` function, the contract applies swap fees and burn fees whenever either the sender or recipient is marked as a swap pair:

```
function _swapAwareTransfer(address sender, address recipient, uint256 amount) internal {
    if (isSwapPair[sender] || isSwapPair[recipient]) {
        uint256 feeAmount  = (amount * swapFeeBP)  / FEE_DENOM;
        uint256 burnAmount = (amount * burnFeeBP) / FEE_DENOM;
        uint256 rest       = amount - feeAmount - burnAmount;
        // ... fee collection logic
    } else {
        // standard no-fee move
        super._transfer(sender, recipient, amount);
    }
}
```

The isSwapPair mapping is a simple boolean flag that cannot differentiate between: - Trading activities - where users swap tokens (should be charged fees). - Liquidity provision - where users add/remove liquidity (should not be charged fees based on the document: "Apply configurable swap and burn fees when the token is traded on designated AMM pairs").

Since the current implementation treats all interactions with swap pairs as trading activities, without considering the specific operation being performed, liquidity providers will be charged swap fees when adding/removing liquidity.

Recommendations

Add fee exemption whitelist.

[M-14] Token mismatch in `claimCollectorRewards()` allows discrepancies

Severity

Impact: Medium

Likelihood: Medium

Description

The `claimCollectorRewards` function accepts a token parameter but **doesn't verify that it matches the actual token transferred by the target contract:**

```
function claimCollectorRewards(address token, address target, bytes32 feeType) external
nonReentrant whenNotPaused returns (bool) {
    if (!isTokenSupported[token]) revert TokenNotSupported();
    if (!isTargetWhitelisted[target]) revert TargetNotWhitelisted();
    if (feeTypes[feeType].feeType == bytes32(0)) revert FeeTypeDoesNotExist();
```

The function measures balance changes for the specified token:

```
uint256 balanceBefore = IERC20(token).balanceOf(address(this));
```

However, the target contract (e.g., `LendingPool`) transfers a specific token regardless of the parameter:

```
// Transfer reserve asset rewards to fee collector
IERC20(reserve.reserveAssetAddress).safeTransfer(feeCollector, claimableAmount);
```

This creates a vulnerability where: - Malicious actors can specify any supported token as the token parameter. - Accounting mismatch occurs when the specified token doesn't match the actual transferred token. - Fee distribution is calculated based on the wrong token's balance change(which is 0). - The actual transferred token is not being recorded properly, causing accounting discrepancies.

For example, if `LendingPool` transfers `reserve.reserveAssetAddress (crvUSD)` but the caller specifies a different supported token (e.g., `USDC`), the function will: - Record fee distribution for USDC (which received no tokens). - Ignore the actual crvUSD received. - Create accounting inconsistencies.

Recommendations

Add validation to ensure the specified token matches what the target contract actually transfers.

[M-15] Uneven distribution in random NFT selection due to empty adapters

Severity

Impact: Low

Likelihood: High

Description

The random NFT selection algorithm in `getNextRandomNFT` uses a circular scan approach that creates a biased probability distribution:

```
function getNextRandomNFT() public view returns (address adapter, uint256 tokenId) {
    uint256 rand = requestStatus.randomWords[0];
    uint256 start = rand % aCount;
    uint256 i = start;
    // circular-scan until we find a non-empty adapter
    do {
        address candidate = redeemableERC721Adapters[i];
        uint256 balance = IERC721VaultAdapter(candidate).getDepositedTokensCount();
        if (balance > 0) {
            adapter = candidate;
            uint256 idx = uint256(keccak256(abi.encode(rand, i))) % balance;
            tokenId = IERC721VaultAdapter(candidate).getDepositedTokenAtIndex(idx);
            return (adapter, tokenId);
        }
        i = (i + 1) % aCount;
    } while (i != start);
    revert("RWAVault: no NFTs available");
}
```

The algorithm starts at a random position and scans until it finds the first non-empty adapter, thus adapters closer to the starting position have a higher selection probability.

For example, with 5 adapters [1,0,0,0,1] (only adapters 0 and 4 have NFTs): - If start = 0: selects adapter 0 (100% chance). - If start = 1,2,3: scans through empty adapters, always selects adapter 4 (100% chance). - If start = 4: selects adapter 4 (100% chance). Result: adapter 0 has 20% chance, adapter 4 has 80% chance.

This violates the fair random selection principle and could create an unfair advantage for certain NFT collections.

Recommendations

First, collect all non-empty adapters, then randomly select from them.

[M-16] Insufficient discount protection check

Severity

Impact: Medium

Likelihood: Medium

Description

In `RWAVault.sol`, the contract attempts to prevent proxy exploitation of the Llama perks discount:

```
// Only apply discount when msg.sender is the same as receiver to prevent proxy exploitation
uint256 discountPercentage = 0;
if (msg.sender == receiver) {
    discountPercentage = _calculateLlamaPerkDiscount(msg.sender);
}
```

However, this protection is insufficient because a smart contract can call `depositAsset` with `msg.sender == receiver` (both being the contract address), then internally distribute the discounted shares to other users.

Thus, the current protection is insufficient.

Recommendations

Add a check of `extcodesize` and `tx.origin` to ensure the user is `EOA`.

[M-17] Staleness check missing in NAV oracle usage

Severity

Impact: Medium

Likelihood: Medium

Description

In `RAACNFTVaultAdapter`, the `totalValue()` function calls `navOracle.getNAV()`, which returns a timestamp along with the NAV value, but this timestamp is completely ignored in the implementation.

```
interface RAACNFTVaultAdapterNAVOracle {
    /// @notice Returns the NAV of the vault in crvUSD (18 decimals)
    /// @dev Use the oracle that will provide the price of the NFTs in the vault (possibly
chainlink external adapter price feed)
    /// @return decimals The decimals of the NAV value
    /// @return value The NAV value
    /// @return timestamp The timestamp of the NAV value
    function getNAV() external view returns (uint8 decimals, uint256 value, uint256 timestamp);
}
```

```
function totalValue() public view override returns (uint256) {
    if (address(navOracle) != address(0)) {
        try navOracle.getNAV() returns (uint8 decimals, uint256 value, uint256) {
            return value * 10 ** (18 - decimals);
        } catch {
            return super.totalValue();
        }
    }

    return super.totalValue();
}
```

However, the function ignores the `timestamp` return value and doesn't perform any staleness validation. This means the vault could use outdated NAV data for critical operations.

Recommendations

Implement staleness checks in the `totalValue()` function:

```solidity function totalValue() public view override returns (uint256) { if (address(navOracle) != address(0)) { try navOracle.getNAV() returns (uint8 decimals, uint256 value, uint256 timestamp) { // Add staleness check if (block.timestamp - timestamp > MAX_STALENESS_DURATION) { return super.totalValue(); } return value * 10 ** (18 - decimals); } catch { return super.totalValue(); } }

```
return super.totalValue();
```

} ``` solidity

## [M-18] Liquidations depend on withdrawals being enabled

### Severity

**Impact:** High

**Likelihood:** Low
```

Description

The StabilityPool liquidation process calls lendingPool.withdraw(rTokenAmountRequired) to obtain crvUSD in order to repay the borrower's debt. However, this function internally checks parameters.withdrawalsPaused and reverts if withdrawals are paused:

`if (parameters.withdrawalsPaused) revert WithdrawalsArePaused();` This creates a critical availability issue:

When withdrawals are paused, liquidations cannot proceed because the system cannot withdraw rToken → crvUSD.

This is dangerous during volatile conditions or emergency pause scenarios where liquidation is most needed to prevent insolvency or cascading failures.

Recommendations

Modify the existing withdraw() to allow trusted roles like StabilityPool to bypass the pause.

[M-19] Blacklisted accounts bypass restrictions by transferring RToken

Severity

Impact: Medium

Likelihood: Medium

Description

While the LendingPool.deposit() function checks if a user is blacklisted via notBlacklisted(msg.sender), the associated RToken contract allows unrestricted transfers, including to and from blacklisted accounts:

```
function _update(address from, address to, uint256 amount) internal override {
    // ...
    super._update(from, to, amount);
}
```

There is no blacklist check in the transfer or _update() function, meaning blacklisted accounts can receive RTokens via transfers, even though they are blocked from direct deposit or withdrawal operations via the LendingPool.

Recommendations

Add blacklist checks inside the RToken's _update() or transfer() logic.

[M-20] Partial yield withdrawals may result in permanent reward loss

Severity

Impact: Medium

Likelihood: Medium

Description

In the VaultProxy.withdrawFromVault function, the protocol calculates and withdraws accrued yield based on share price changes. If not all yields are withdrawable, it just uses the max withdraw amount. However, it updates lastSharePrice even when not all the yield is withdrawn, which may cause the remaining yield to be permanently unaccounted for:

```
   if (totalYield > 0) {
            uint256 maxWithdrawable = _maxWithdraw(address(this));
            uint256 yieldToWithdraw = totalYield > maxWithdrawable ? maxWithdrawable :
totalYield;

            if (yieldToWithdraw > 0) {
                vaultRewards.unclaimedRewards += yieldToWithdraw;
                _withdraw(yieldToWithdraw, address(this), address(this));
            }
        }
```

If not all of the calculated yield is withdrawable due to liquidity constraints (maxWithdrawable < totalYield), then only a portion of the accrued yield is withdrawn.

Yet, the entire yield window is closed by updating lastSharePrice to the current one.

Thus, the unclaimed (but unwithdrawn) portion of yield is never revisited, and the next harvest starts from a higher lastSharePrice, skipping over the yield that wasn't pulled out.

Recommendations

Only update lastSharePrice and unclaimedRewards if all of the expected yield was withdrawn:

[M-21] Immediate withdrawals are possible by bypassing timelock with token transfer

Severity

Impact: Medium

Likelihood: Medium

Description

The _executeWithdrawTimelock() function includes a check intended to prevent users from depositing and withdrawing in the same block, which is a common pattern in flash loan or sandwich attack mitigation.

```
if (depositBlock[user] == block.timestamp) revert CannotWithdrawInSameBlock();
```

However, this protection can be bypassed through RToken transfers:

- User A deposits reserve assets in block N, receiving RTokens. depositBlock[A] = N.
- In the same block, user A transfers the RTokens to user B.
- User B has depositBlock[B] == 0, because they haven't called deposit().
- If user B had already queued a requestWithdraw(amount) earlier, they can now call withdraw() immediately in the same block, successfully bypassing the depositBlock restriction.

Recommendations

Prevent RToken transfers in the same block where they were minted.

[M-22] Oracle update front-running for profit

Severity

Impact: High

Likelihood: Low

Description

The RAACHousePriceOracle utilizes Chainlink Functions to fetch house pricing data from off-chain APIs. This process involves two separate transactions with a potential delay of up to 5 minutes between request initiation and fulfillment. This design creates a vulnerability where attackers can monitor the mempool to preview incoming price updates and execute profitable front-running attacks.

When the Chainlink node calls `handleOracleFulfillment`, the new price data is visible in the mempool before it's committed on-chain, creating an exploitable window of opportunity.

```
function sendRequest(
    string calldata source,
    FunctionsRequest.Location secretsLocation,
    bytes calldata encryptedSecretsReference,
    string[] calldata args,
    bytes[] calldata bytesArgs,
    uint64 subscriptionId,
    uint32 callbackGasLimit
) external virtual onlyOwner returns (bytes32) {
    ...
```

```
        bytes32 requestId = _sendRequest(
            encodedRequest,
            subscriptionId,
            callbackGasLimit,
            donId
        );

        ...
    }


    function fulfillRequest(
        bytes32 requestId,
        bytes memory response,
        bytes memory err
    ) internal override {
        ...
        if (err.length == 0) {
            if (response.length == 0) {
                revert FulfillmentFailed();
            }
            _processResponse(requestId, response); // Attackers front-run before this executes
        }

        ...
    }
```

Consider scenarios:

- Front-Running an NFT Mint: RAACNFT # 123 is priced at $100,000 on-chain. An attacker observes a pending oracle transaction that will update the price to $120,000.
  - Attacker monitors mempool and decodes the `handleOracleFulfillment` transaction.
  - Attacker immediately submits a high-priority transaction calling RAACNFT.mint(123).
  - Attacker pays current price of $100,000 (plus fees).
  - Oracle update executes, setting new price to $120,000.
  - Result: Attacker gains $20,000 instantly, risk-free profit.
- Front-Running an RWAVault deposit: Attacker holds RAACNFT tokens worth $100,000. They observe the price will decrease to $80,000.
  - Attacker detects an incoming negative price update in mempool.
  - Attacker calls `RWAVault.depositAsset` with maximum gas priority.
  - Deposit executes at old price, minting index tokens at inflated rate.
  - Price update executes, reducing RAACNFT value by 20%.
  - Result: Attacker socializes $20,000 loss to all index token holders.

Recommendations

It's recommended to: - add a cooldown period that starts at sendRequest and ends after fulfillRequest is complete to allow user mints/deposits OR. - Combine the Chainlink Functions service with private transaction relay (like Flashbots).

[M-23] Supply cap not enforced due to borrow impact and ignored interest

Severity

Impact: Medium

Likelihood: Medium

Description

The `LendingPool` implements a `supplyCap` mechanism to limit the total amount of assets supplied into each reserve. However, the enforcement logic inside `_validateDepositSupplyCap` relies on the `reserve.totalLiquidity` value, which introduces two critical weaknesses:

Supply Cap Bypass via Borrow-Induced Liquidity Reduction

The `reserve.totalLiquidity` metric **decreases when users borrow**, as it tracks the immediately available liquidity. This allows malicious users to actively bypass the supply cap by manipulating the reserve's liquidity through borrowing.

Attack scenario:

1. The attacker borrows assets from the pool, reducing `totalLiquidity`.
2. With the artificially lowered `totalLiquidity`, the attacker then deposits additional funds, effectively exceeding the intended `supplyCap`.
3. Finally, the attacker repays their borrow, restoring the liquidity — but the deposits now exceed the cap.

This exploit makes the cap mechanism ineffective since it is calculated on a value that can be actively reduced by the same users who wish to deposit more.

Example from implementation:

```
function _validateDepositSupplyCap(uint256 amount) internal view {
    uint256 totalSupplied = reserve.totalLiquidity;  // ← decreases on borrow
    if (totalSupplied + amount > parameters.supplyCap) revert SupplyCapReached();
}

function updateInterestRatesAndLiquidity(...) internal {
    if (liquidityTaken > 0) {
        reserve.totalLiquidity = reserve.totalLiquidity - liquidityTaken.toUint128();  // ←
```

```
reduces on borrow
    }
}
```

Underestimation by Ignoring Accrued Interest

In addition to being reduced by borrows, `reserve.totalLiquidity` **does not account for interest accrued on outstanding debts**, which means the actual total obligation in the system is underreported. As interest accumulates over time, the effective pool size can grow well beyond the intended cap, simply because the accrued debt interest is not included in `totalLiquidity`.

This enables users to deposit beyond the `supplyCap` even without any active manipulation. The cap enforcement is fundamentally incomplete because it does not consider the full debt-adjusted supply.

Example from implementation:

```
function _validateDepositSupplyCap(uint256 amount) internal view {
    ...
    uint256 totalSupplied = reserve.totalLiquidity;  // ← ignores accrued interest
    if (totalSupplied + amount > parameters.supplyCap) revert SupplyCapReached();
}
```

Impact

- Users can deliberately manipulate the liquidity by borrowing to circumvent supply caps, leading to **uncontrolled growth of the pool beyond configured risk parameters**.
- Even in normal operation, accumulated interest on outstanding borrowings allows the total supplied value to silently surpass the cap, violating intended protocol constraints.

Recommendation

Use a comprehensive metric for cap enforcement:

- Replace `reserve.totalLiquidity` with a metric that accurately reflects the **total deposited supply plus accrued interest**, independent of liquidity changes due to borrowing.
- This could involve tracking a dedicated `totalDeposits` or `totalPrincipal` variable that increments only on deposits and grows via interest indexing, unaffected by borrowings actions.
- Additionally, consider integrating `totalDebt` growth to fully enforce caps on the combined supplied and accrued positions.

This ensures that the `supplyCap` accurately limits the overall exposure of the protocol, preventing both intentional bypasses via borrowing and unintended breaches due to accrued interest.

Low findings

[L-01] Liquidity value after withdraw in `ensureLiquidity()` is wrong

`ensureLiquidity` incorrectly includes the `address(this)` balance (which contains `unclaimedRewards`) when calculating `availableLiquidity` during its operation.

```
    function ensureLiquidity(uint256 amount) external onlyProxy() {
        // if vault is not set, do nothing
        if (address(vault) == address(0)) {
            return;
        }

        uint256 availableLiquidity =
IERC20(reserve.reserveAssetAddress).balanceOf(reserve.reserveRTokenAddress);
        // if we do not have enought, then we need to withdraw from the vault
        if (availableLiquidity < amount) {
            uint256 requiredAmountToWithdraw = amount - availableLiquidity;
            uint256 maxWithdrawable = _maxWithdraw(address(this));

            // Check if we can fulfill the entire withdrawal
            if (maxWithdrawable < requiredAmountToWithdraw) {
                revert("Not enough liquidity to fulfill the withdrawal");
            }

            uint256 totalDeposits = reserve.totalLiquidity;
            uint256 remainingDeposits = totalDeposits - amount;

            // If this is a complete withdrawal (or close to it), ignore buffer requirements
            uint256 newDesiredBuffer;
            if (remainingDeposits > 0) {
                newDesiredBuffer =
remainingDeposits.percentMul(parameters.liquidityBufferRatio);
                // newDesiredBuffer = 0;
            // } else {
            }

            uint256 withdrawFromVaultAmount = requiredAmountToWithdraw;
            if (withdrawFromVaultAmount > maxWithdrawable) {
                withdrawFromVaultAmount = maxWithdrawable;
            }

            if (withdrawFromVaultAmount > 0) {
                withdrawFromVault(withdrawFromVaultAmount);
            }

            // Verify we have enough after withdrawal
>>>         availableLiquidity =
IERC20(reserve.reserveAssetAddress).balanceOf(reserve.reserveRTokenAddress) +
IERC20(reserve.reserveAssetAddress).balanceOf(address(this));
            if (availableLiquidity < amount) {
```

```
                revert("Insufficient liquidity after vault withdrawal");
            }
        }
```

This causes the available liquidity check to pass incorrectly, even though the actual balance available for withdrawal in `reserveRTokenAddress` is insufficient.

[L-02] Pausing repay may cause liquidation risk when pool is unpaused

`LendingPool` can be paused, affecting all operations, including `repay`. During the paused state, interest continues to accrue over time, which may cause users to become liquidatable once the pool is unpaused.

Consider triggering a reserve state update when the pool is paused, and updating `lastUpdateTimestamp` to the latest timestamp without modifying reserve data when unpause is called. This would effectively pause interest accrual while `LendingPool` is paused.

[L-03] `amountScaled` calculation in `RToken.mint` is incorrect

When `mint` is called, it calculates amountScaled as `amount.rayMul(index)`. This is incorrect, as the provided amount should be treated as a scaled amount based on the latest index.

```
    function mint(
        address caller,
        address onBehalfOf,
        uint256 amount,
        uint256 index
    ) external override onlyLendingPool returns (uint256, uint256, uint256) {

        if (caller == address(0) || onBehalfOf == address(0)) revert InvalidAddress();
        if (amount == 0) {
            return (0, 0, 0);
        }

        // uint256 amountScaled = amount.rayDiv(index);

>>>     uint256 amountScaled = amount.rayMul(index);

        uint256 rawBalance = super.balanceOf(onBehalfOf);
        uint256 userIndex = ILendingPool(_lendingPool).getNormalizedIncome() -
_userState[onBehalfOf].index;
        uint256 scaledBalance = rawBalance.rayMul(userIndex);


        // uint256 scaledBalance = balanceOf(onBehalfOf);

        uint256 scaledBalanceIncrease = 0;
        if (_userState[onBehalfOf].index != 0 && _userState[onBehalfOf].index < index) {
            scaledBalanceIncrease = rawBalance.rayMul(index) -
rawBalance.rayMul(_userState[onBehalfOf].index);
```

```
        }

        _userState[onBehalfOf].index = index.toUint128();

        uint256 amountToMint = amount + scaledBalanceIncrease;

        // Update raw total deposits with only the new deposit amount (not the interest)
        _rawTotalDeposits += amount;

        _mint(onBehalfOf, amountToMint.toUint128());

        emit Mint(caller, onBehalfOf, amountToMint, index);

>>>     return (amountToMint, totalSupply(), amountScaled);
    }
```

```
    function deposit(ReserveData storage reserve, ReserveRateData storage rateData, uint256
amount, address depositor) internal returns (uint256 amountMinted) {
        if (amount < 1) revert InvalidAmount();

        // Update reserve interests
        updateReserveInterests(reserve, rateData);

        // Transfer asset from caller to the RToken contract
        IERC20(reserve.reserveAssetAddress).safeTransferFrom(
            depositor,                    // from
            reserve.reserveRTokenAddress,  // to
>>>         amount                         // amount
        );

        // Mint RToken to the depositor (scaling handled inside RToken)
>>>     (uint256 amountScaled, uint256 newTotalSupply, uint256 amountUnderlying) =
IRToken(reserve.reserveRTokenAddress).mint(
            address(this),         // caller
            depositor,             // onBehalfOf
            amount,                // amount
            reserve.liquidityIndex // index
        );

        amountMinted = amountScaled;

        // Update the total liquidity and interest rates
        updateInterestRatesAndLiquidity(reserve, rateData, amount, 0);

        emit Deposit(depositor, amount, amountMinted);

        return amountMinted;
    }
```

Calculating `amount.rayMul(index)` results in an `amountUnderlying` greater than the `amount` transferred by the user.

Instead, set `amountScaled` to the provided `amount`.

[L-04] `Rwavault's` `depositAsset()` and `poolDepositAsset()` return wrong shares

When `_deposit` is called, the actual shares received by users are `netAmount` (shares after fees). Returning `sharesMinted` (shares before fees) instead could cause users to process an incorrect amount of minted shares.

```
    function _deposit(address adapter, bytes calldata data, address receiver, uint256
mintFeePercentage) internal returns(uint256) {
        if (receiver == address(0)) revert InvalidAddress();

        uint256 assetsBefore = totalAssets();
        uint256 supplyBefore = IVaultToken(vaultToken).totalSupply();
        uint256 depositedAssetValue = IVaultAssetAdapter(adapter).deposit(data, msg.sender);
        uint256 sharesMinted;
        if (supplyBefore == 0 || assetsBefore == 0) {
            sharesMinted = depositedAssetValue;  // 1:1 for the first depositor
        } else {
            sharesMinted = (depositedAssetValue * supplyBefore) / assetsBefore;
        }

        if (sharesMinted == 0) revert InvalidShares();

        // Mint all in current contract, then distribute after the fees
        IVaultToken(vaultToken).mint(address(this), sharesMinted);
        uint256 mintingFee = 0;

        if (feeCollector != address(0) && mintFeePercentage > 0) {
            mintingFee = (sharesMinted * mintFeePercentage) / 1e4;
            // Take some fees
            // send minting fees to fee collector
            bool approval = IERC20(vaultToken).approve(feeCollector, mintingFee);
            if (!approval) revert ApprovalFailed();
            IFeeCollector(feeCollector).collectFee(vaultToken, address(this), mintingFee,
keccak256("MINT_FEE"));
        }

        // Send remaining to the user
>>>     uint256 netAmount = sharesMinted - mintingFee;
        IERC20(vaultToken).transfer(receiver, netAmount);
>>>     return sharesMinted;
    }
```

Return `netAmount` instead of `sharesMinted`.

[L-05] Incorrect return value in `getNormalizedDebt()` if time not passed

The `getNormalizedDebt` function incorrectly returns `reserve.totalUsage` instead of `reserve.usageIndex` when no time has elapsed since the last update (timeDelta < 1).

```
    function getNormalizedDebt(ReserveData storage reserve, ReserveRateData storage rateData)
internal view returns (uint256) {
        uint256 timeDelta = block.timestamp - uint256(reserve.lastUpdateTimestamp);
        if (timeDelta < 1) {
            return reserve.totalUsage;
        }

        return calculateCompoundedInterest(rateData.currentUsageRate,
timeDelta).rayMul(reserve.usageIndex);
    }
```

[L-06] Blacklisted users bypass withdrawal limits if timelock is disabled

When `withdrawTimelockDuration` is set to 0, blacklisted users can circumvent withdrawal restrictions by transferring their RToken to a non-blacklisted address and withdrawing from that address. This occurs because RToken transfers are not subject to blacklist restrictions, while the `LendingPool.withdraw` function only checks if `msg.sender` is blacklisted.

```
    function withdraw(uint256 amount) external nonReentrant whenNotPaused
onlyValidAmount(amount) notBlacklisted(msg.sender) {
        ...
    }
```

Consider preventing blacklisted users from transferring Rtoken.

[L-07] Missing asset balance validation in vault adapter unregistration

The `unregisterAdapter` function in RWAVault allows the owner to remove an adapter from the supported adapters list without verifying that the adapter has zero assets. This creates an accounting inconsistency where assets remain locked in the adapter while being excluded from the vault's total asset calculations. It leads to deflating the vault token price.

```
    function unregisterAdapter(address adapter) external onlyOwner {
        require(supportedAdapters[adapter], "RWAVAult: not supported");
        supportedAdapters[adapter] = false;
        // Remove from adapters array
        for (uint256 i = 0; i < adapters.length; i++) {
            if (adapters[i] == adapter) {
                adapters[i] = adapters[adapters.length - 1];
                adapters.pop();
                break;
            }
        }
        emit AdapterRemoved(adapter);
    }
```

It's recommended to add a balance check before allowing adapter unregistration.

[L-08] No price staleness check in `RAACNFT` minting

The `mint` function in the RAACNFT contract does not validate the freshness of price data retrieved from the `RAACHousePrices` oracle, potentially allowing users to mint NFTs using stale price information that may not reflect current market conditions.

When users call the `mint` function to mint RAAC NFTs, the contract retrieves the current house price from the RAACHousePrices oracle without checking when this price was last updated:

```
    function mint(uint256 _tokenId, uint256 _amount) public override nonReentrant
notBlacklisted(msg.sender) {
        (uint256 price, ) = raac_hp.getLatestPrice(_tokenId);
        ...
    }
```

It's recommended to implement a staleness threshold check in the `mint` function.

[L-09] Collectors allow blacklisted user interaction

The protocol's collector contracts (`ERC20Collector` , `NFTRoyaltyFeeCollector` , and `FeeCollector`) do not inherit from the `WithCompliance` contract. This allows blacklisted users to continue interacting with fee collection mechanisms, undermining the protocol's compliance framework.

```
contract ERC20Collector is AccessControl, ReentrancyGuard
contract NFTRoyaltyFeeCollector is Ownable, ReentrancyGuard
contract FeeCollector is IFeeCollector, AccessControl, ReentrancyGuard, Pausable
```

It's recommended to block blacklist users from interacting with the collectors.

[L-10] Blacklisted users can withdraw from `StabilityPool`

The `withdraw` function in the `StabilityPool` contract lacks blacklist enforcement, allowing previously blacklisted addresses to withdraw RTokens. This creates a potential bypass mechanism for blacklist restrictions.

Consider a scenario:

- User deposits RTokens into the StabilityPool.
- User initiates a withdrawal request.
- User gets blacklisted due to malicious activity or regulatory requirements.
- User can still execute the withdrawal despite being blacklisted.
- Blacklisted user successfully extracts RTokens from the protocol.

```
    function withdraw(uint256 deTokenAmount) external nonReentrant whenNotPaused
validAmount(deTokenAmount) {
        ...
    }
```

It's recommended to add the `notBlacklisted` modifier to the withdraw function.

[L-11] Protocol uses old Curve Liquidity Gauge V5 interface

The protocol's liquidation strategy currently interfaces with Curve Liquidity Gauge V5 when depositing reward tokens, while Curve has released the newer V6 gauge. New gauge deployments via `CurveTwoCryptoFactory.deploy_gauge()` now create V6 gauges by default.

https://docs.curve.finance/liquidity-gauges-and-minting-crv/gauges/LiquidityGaugeV6/.

```
    function _handleLiquidityRewards(uint256 amount) internal {
        ...
        ICurveLiquidityGaugeV5(liquidityGauge).deposit_reward_token(underlyingVaultToken,
amount);
    }
```

It's recommended to update the interface import and usage to Curve Liquidity Gauge V6.

[L-12] Actual withdrawal amounts not validated against expected amounts

```
    function withdrawFromVault(uint256 amount) public onlyProxy {
        uint256 maxWithdrawable = _maxWithdraw(address(this));
        ...

        uint256 actualWithdrawal = idealWithdrawal > maxWithdrawable ? maxWithdrawable :
idealWithdrawal;

        _withdraw(actualWithdrawal, address(this), address(this));

        ...
    }
```

When withdrawing from the vault, the `maxWithdraw` function returns the maximum withdrawable amount but does not account for unrealized losses in the returned value. This creates a scenario where a user might call `withdraw(100)` but only receive 90 assets due to a 10% unrealized loss.

The `scrvUSD` documentation explicitly acknowledges this behavior:

```
def _max_withdraw(
    owner: address,
    max_loss: uint256,
    strategies: DynArray[address, MAX_QUEUE]
) -> uint256:
```

```
    """
    @dev Returns the max amount of `asset` an `owner` can withdraw.

    This will do a full simulation of the withdraw in order to determine
    how much is currently liquid and if the `max_loss` would allow for the
    tx to not revert.

    This will track any expected loss to check if the tx will revert, but
    not account for it in the amount returned since it is unrealised and
    therefore will not be accounted for in the conversion rates.

    i.e. If we have 100 debt and 10 of unrealised loss, the max we can get
    out is 90, but a user of the vault will need to call withdraw with 100
    in order to get the full 90 out.
    """
    ...
```

https://github.com/curvefi/scrvusd/blob/main/contracts/yearn/VaultV3.vy#L536-L555.

Currently mitigated for scrvUSD vault (no strategies deployed), but poses risk for future integrations with strategy-enabled vaults.

It's recommended to implement validation to check actual received amounts against expected amounts whenever _max_withdraw is used.

[L-13] Users can bypass fees by opening vaults and depositing

The LendingPool contract fails to initialize the `vaultOpeningFee` and `depositFee` parameters in its constructor, creating a window of opportunity where users can bypass protocol fees entirely. During the period between contract deployment and fee parameter initialization, users can:

- Call `openVaultPosition` without paying the required vault opening fee.
- Call `deposit` without paying the deposit fee.

This vulnerability stems from the Parameters struct containing uninitialized fee values (defaulting to 0) when the contract is first deployed.

It's recommended to initialize fee parameters in the constructor to prevent the bypass window.

[L-14] Storage gap absent in `StabilityPoolStorage` risks upgrade collision

The `StabilityPoolStorage` contract lacks a storage gap, creating a risk of storage slot collisions when new variables are added during contract upgrades. This vulnerability stems from the complex inheritance hierarchy where `StabilityPool` inherits from multiple contracts that define storage variables in specific slots.

```
contract StabilityPoolStorage is IStabilityPoolStorage {
    IRToken public rToken;                          // slot 0
    IDEToken public deToken;                        // slot 1
    // ... additional variables ...
    mapping(address => uint256) public depositBlock; // slot 18
    // NO STORAGE GAP - slots 19+ are unprotected
}

abstract contract ReentrancyGuard {
    uint256 private _status;                        // slot 19
}

contract StabilityPool is StabilityPoolStorage, ... {
    address public liquidationStrategyProxy;        // slot 20
}
```

If new storage variables are added to `StabilityPoolStorage` during an upgrade, `ReentrancyGuard._status` and `StabilityPool.liquidationStrategyProxy` would be overwritten. It leads to storage collision.

It's recommended to add a storage gap to `StabilityPoolStorage` to reserve slots for future upgrades.

[L-15] `DebtToken` decimal precision mismatch with underlying assets

The `DebtToken` contract inherits from OpenZeppelin's ERC20 implementation, which defaults to 18 decimals. However, when the lending pool supports assets with fewer than 18 decimals, this creates a precision mismatch between the debt token representation and the actual borrowed asset's native precision.

It's recommended to modify `DebtToken` to inherit decimals from the underlying asset.

[L-16] Inconsistent same block withdrawal protection implementation

In LendingPool, the variable `depositBlock` and its documentation indicate it should track block numbers, but the code implementation uses `block.timestamp` throughout. In StabilityPool, the `block.number` is also used.

```
    /// @notice track the block number when the user deposited
    mapping(address => uint256) public depositBlock;

    function deposit(uint256 amount) external nonReentrant whenNotPaused
onlyValidAmount(amount) notBlacklisted(msg.sender) {
        ...

        depositBlock[msg.sender] = block.timestamp;

        emit Deposit(msg.sender, depositAmount, mintedAmount);
```

```
    }


    function _executeWithdrawTimelock(address user, uint256 amount) internal {
        if (depositBlock[user] == block.timestamp) revert
CannotWithdrawInSameBlock(); // @audit use not consistent with stability pool
        ...
    }
```

It's recommended to update to `block.number` for consistency.

[L-17] Missing validation in `RAACHousePriceOracle`

The `_processResponse` function in RAACHousePriceOracle accepts and processes oracle responses without validating the returned price value. This allows invalid prices (including zero) to be set directly in the RAACHousePrices contract, leading to incorrect liquidations and index token pricing.

```
    function _processResponse (bytes32 requestId, bytes memory response) internal override {
        uint256 price = abi.decode(response, (uint256));
        uint256 houseId = requestToHouseId[requestId];
        housePrices.setHousePrice(houseId, price); // @audit doesn't validate the returned price
        delete requestToHouseId[requestId];
        emit HousePriceUpdated(houseId, price);
    }
```

It's recommended to add price validation before calling `setHousePrice`.

[L-18] Treasury address can be updated immediately despite defined delay constant

The `FeeCollector` contract defines a `TREASURY_UPDATE_DELAY` constant of 1 day but fails to implement this delay mechanism when updating the treasury address through the `setTargetAddress` function. This allows treasury address changes to take effect immediately, potentially undermining governance processes.

```
uint256 public constant TREASURY_UPDATE_DELAY = 1 days;

function setTargetAddress(bytes32 target, address newAddress) external
onlyRole(FEE_MANAGER_ROLE) {
    ...

    if (target == TREASURY) {
        treasuryAddress = newAddress;
    ...

}
```

It's recommended to implement the delay mechanism or remove the unused constant.

[L-19] Incorrect total supply returned on zero-amount mint

The RToken.mint function returns `(0, 0, 0)` when called with `amount = 0`, but should return the actual `totalSupply` ()

```
    function mint(
        address caller,
        address onBehalfOf,
        uint256 amount,
        uint256 index
    ) external override onlyLendingPool returns (uint256, uint256, uint256) {
        if (caller == address(0) || onBehalfOf == address(0)) revert InvalidAddress();
        if (amount == 0) {
            return (0, 0, 0);
        }
```

It's recommended to return `(0, totalSupply(), 0)` to ensure consistent total supply reporting across all mint operations.

[L-20] Rounding error in percentMul causes fee collection reversion

The `FeeCollector._updateCollectedFees` function allocates a fee `amount` among up to six recipients based on pre-defined shares. The calculation for each share uses `PercentageMath.percentMul`, which rounds the result up if the fractional part is 0.5 or greater.

When multiple shares are calculated this way from a specific `amount`, the cumulative effect of rounding can cause the sum of the shares (`totalTax`) to be greater than the original `amount`. This leads to an arithmetic underflow in the line `uint256 extra = amount - totalTax;`, causing the entire `collectFee` transaction to revert. This creates a DOS condition for the fee distribution mechanism.

```
    function _updateCollectedFees(address token, bytes32 feeType, uint256 amount) internal {
        FeeType memory feeTypeData = feeTypes[feeType];
        uint256 treasuryAmount =
amount.percentMul(feeTypeData.treasuryShare); // can be rounded up
        uint256 repairFundAmount = amount.percentMul(feeTypeData.repairFundShare);
        uint256 burnAmount = amount.percentMul(feeTypeData.burnShare);
        uint256 veRAACTokenAmount = amount.percentMul(feeTypeData.veRAACTokenShare);
        uint256 raacCorpAmount = amount.percentMul(feeTypeData.raacCorpShare);
        uint256 otherAmount = amount.percentMul(feeTypeData.otherShare);

        uint256 totalTax = treasuryAmount + repairFundAmount + burnAmount + veRAACTokenAmount +
raacCorpAmount + otherAmount;

        if (totalTax != amount) {
            uint256 extra = amount - totalTax; // revert if amount < totalTax
            treasuryAmount += extra;
        }
        ...
    }
```

Consider a scenario:

Assume a fee split where shares sum to 100% (`10000` basis points): - 5 recipients: 1667 basis points each (16.67%). - 1 recipient: 1665 basis points (16.65%).

If `collectFee` is called with an `amount` of 999: - Share 1-5: 999 × 1667 ÷ 10000 = 166.5333 → rounds to 167. - Share 6: 999 × 1665 ÷ 10000 = 166.3335 → rounds to 166. - totalTax = (5 × 167) + 166 = 1001. - extra = 999 - 1001 = underflow (revert).

It's recommended to calculate using simple division, for example: repairFundAmount = (amount * feeTypeData.repairFundShare) / BASIS_POINTS.

[L-21] Use `ReentrancyGuardUpgradeable` in `StabilityPool`

StabilityPool is an upgradeable contract, it should use `ReentrancyGuardUpgradeable` and call `__ReentrancyGuard_init` in the contract initialization.

```
contract StabilityPool is StabilityPoolStorage, IStabilityPool, Initializable, ReentrancyGuard,
OwnableUpgradeable, PausableUpgradeable, ERC721Holder, WithCompliance
```

[L-22] Immediate liquidation via grace period modification

The `setParameter()` function in `LendingPool.sol` allows the owner to modify the `liquidationGracePeriod` without considering the impact on existing positions that are already under liquidation.

```
        } else if (param == OwnerParameter.LiquidationGracePeriod) {
            if (newValue < 1 days || newValue > 7 days) revert InvalidGracePeriod();
            oldValue = parameters.liquidationGracePeriod;
            parameters.liquidationGracePeriod = newValue;
```

When the grace period is reduced, positions that were previously within their grace period(could be re-paid or closed) may immediately become eligible for liquidation, creating an unfair situation for users who relied on the original grace period terms.

```
        if (block.timestamp > position.liquidationStartTime + parameters.liquidationGracePeriod
&& positionDebt > 0) {
            revert GracePeriodExpired();
        }
```

I see this issue as acknowledged in the previous audit, but I want to raise this, since this should be properly handled to reduce unexpected behavior.

[L-23] Unnecessary approval reset before revert in `distributeFees()`

The `distributeFees` function performs an unnecessary `approve(0)` call before reverting when an external call fails. Since the function immediately reverts after the failed call, the approval reset serves no purpose and wastes gas.

```
if (!success) {
    // Remove approval on failure
    IERC20(token).approve(targetAddress, 0);
    revert DistributionFailed();
}
```

When the external call fails, the entire transaction is reverted, making the approval reset unnecessary.

[L-24] Missing token support validation in `distributeFees()`

The `distributeFees` function in `FeeCollector` performs token transfers and approvals without validating that the token is supported, unlike other functions in the same contract that handle tokens:

```
function distributeFees(bytes32 feeType, bytes32 target, address token) external whenNotPaused
onlyRole(DISTRIBUTOR_ROLE) {
    // If a fee type is not set or target is not set, we don't distribute
    if (feeTypes[feeType].feeType == bytes32(0)) revert FeeTypeDoesNotExist();
```

The function only checks for valid fee types and targets, but omits the token support validation that is present in other functions:

```
if (!isTokenSupported[token]) revert TokenNotSupported();
```

Without the token support check, the function could potentially distribute unsupported tokens that may have unexpected behavior.

[L-25] Minting fee bypass via small deposit rounding

The minting fee calculation in the `RWAVault::_deposit` function uses integer division that rounds down:

```
mintingFee = (sharesMinted * mintFeePercentage) / 1e4;
```

Attacker deposits a minimal amount that results in `sharesMinted * mintFeePercentage < 1e4`, as a result, the intended `2%` minting fee is bypassed.

This can be combined with `LlamaPerkDiscount`, which makes the rounding down much easier.

```
        // Only apply discount when msg.sender is the same as receiver to prevent proxy
exploitation
        uint256 discountPercentage = 0;
        if (msg.sender == receiver) {
            discountPercentage = _calculateLlamaPerkDiscount(msg.sender);
        }
        uint256 mintingFee = (indexTokenMintingFee * (100_00 - discountPercentage)) / 100_00;
```

[L-26] Inflation attack in `RWAVault` share calculation via admin actions

The share calculation in `_deposit` function uses the formula:

```
sharesMinted = (depositedAssetValue * supplyBefore) / assetsBefore;
```

This formula assumes that `supplyBefore` and `assetsBefore` maintain a consistent relationship. However, two admin functions can break this assumption: - `adminDepositAsset` : Increases `assetsBefore` without minting any vault tokens. - `burnVaultToken` : Decreases `supplyBefore` without reducing underlying assets.

```
    /// @inheritdoc IBaseHybridVault
    function adminDepositAsset(address adapter, bytes calldata data) external onlyManager
onlySupportedAdapter(adapter) nonReentrant {
        // deposit but do not mint vault token
        IVaultAssetAdapter(adapter).deposit(data, msg.sender);
        emit AdminDeposit(adapter, data);
    }
```

```
    /// @inheritdoc IBaseHybridVault
    function burnVaultToken(address from, uint256 amount) external {
        if (!isManager(msg.sender) && msg.sender != stabilityPool) revert("only manager or
stability pool");
        IVaultToken(vaultToken).burn(from, amount);
    }
```

This can be utilized by attacker to perform an inflation attack, for example: - Attacker deposits a minimal amount (1 wei) to initialize the vault. - Manager calls `adminDepositAsset` to add significant assets without minting shares. - Result: `assetsBefore` is high, `supplyBefore` is low. - Subsequent depositors receive inflated shares due to the manipulated ratio.

[L-27] Unsafe NFT transfers without receiver check

In `ERC721VaultAdapter` , the withdrawal logic is:

```
token.transferFrom(address(this), to, tokenId);
emit TokenWithdrawn(address(token), to, tokenId);
```

Using `transferFrom` bypasses the `ERC721` receiver check, so if `to` is a contract that does not implement `onERC721Received` , the NFT will be locked and unrecoverable. This is a common source of asset loss in NFT protocols.

To mitigate this issue, always try using `safeTransferFrom`.

[L-28] Redundant role admin assignment in `ComplianceRegistry`

In `ComplianceRegistry`, the following line appears in the constructor or initialization logic:

```
    constructor() {
        _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
        _grantRole(BLACKLIST_ADMIN_ROLE, msg.sender);

        // Make DEFAULT_ADMIN_ROLE the admin of BLACKLIST_ADMIN_ROLE
        _setRoleAdmin(BLACKLIST_ADMIN_ROLE, DEFAULT_ADMIN_ROLE);
    }
```

However, OpenZeppelin's `AccessControl` already assigns `DEFAULT_ADMIN_ROLE` as the admin for all roles by default.

[L-29] Low VRF confirmations increase reorg risk on unstable chains

According to the sponsor, "We had multi-chains expectations for a long time, it was recently reduced from immediate scope, but mid/long term, all EVMs could/should be potentially a target.'

In BaseVRFv2Consumer.sol, the VRF request confirmation count is set as follows:

```
uint16 requestConfirmations = 3;
```

A low confirmation count (such as 3) is insufficient for chains with frequent or deep reorganizations (e.g., Base, Polygon, Arbitrum). This can result in VRF results being reverted or manipulated if a reorg occurs, undermining the security and reliability of randomness.

[L-30] Incorrect contract detection risks fund loss Post-EIP-7702

The following code is used to check if an address is a contract:

```
bool isContract = size > 0;
if (isContract) {
    // call targetAddress
}
```

With EIP-7702, EOAs can have code during a transaction, making `extcodesize` return a nonzero value. However, some EOAs may not implement the expected contract interface or logic(they may delegate to another EOA and still perform like an EOA).

If the contract proceeds to call such an address, the call will succeed (since `EOAs` can accept calls), but no contract logic will be executed. This can result in funds or actions being locked in the contract without being sent out.

```
            if (!success) {
                // Remove approval on failure
                IERC20(token).approve(targetAddress, 0);
                revert DistributionFailed();
            }
```

[L-31] ERC20 `approve` misuse may fail fee distribution

The `ERC20Collector` contract uses the standard `IERC20(underlyingToken).approve(feeCollector, balance)` call and checks its boolean return value. However, not all ERC20 tokens strictly follow the ERC20 standard—some do not return a boolean value for approve, causing the call to revert or behave unexpectedly.

```
bool approve = IERC20(underlyingToken).approve(feeCollector, balance);
if (!approve) revert ApprovalFailed();
```

Some widely used tokens (e.g., `USDT`) do not return a value, which can cause the transaction to revert or the check to fail, even if the approval succeeded.

[L-32] Incorrect operation order in `depositIntoVault()`

The depositIntoVault() function performs operations in the wrong order. `_maxDeposit()` is called before `_harvestYield()`. `_harvestYield()` withdraws assets from the vault, possibly increasing available capacity. These would lead to a deposit less than max available amount.

Fix the operation order by harvesting yield before checking deposit limits:

```
function depositIntoVault(uint256 amount) internal {
    if (amount == 0) return;

    // 1. FIRST harvest yield to get accurate vault state
    _harvestYield();

    // 2. THEN check maximum deposit limit with current state
    uint256 maxDeposit = _maxDeposit(address(this));
    if (amount > maxDeposit) {
        amount = maxDeposit;
    }
```

[L-33] `detoken.totalSupply()` multiplies interest twice

The deToken.totalSupply() function incorrectly applies interest scaling again on top of an already interest-adjusted value, leading to inflated total supply reporting. However, _scaledTotalSupply is not a raw, unscaled value. It is already updated based on interest earned by users, via the _update() function. So when totalSupply() multiplies _scaledTotalSupply again by currentIndex, it effectively applies the index-based interest twice.

Track _scaledTotalSupply interest index and apply the index difference to get the total supply.

[L-34] Frequent small liquidation updates reduce rate gauge

ch time _handleLiquidityRewards() is triggered during liquidation, it calls the deposit_reward_token() function on a Curve-style reward gauge contract. This updates the reward distribution rate and extends the period_finish time.

```
if block.timestamp >= period_finish:
    self.reward_data[_reward_token].rate = _amount / WEEK
else:
    leftover = remaining * self.reward_data[_reward_token].rate
    self.reward_data[_reward_token].rate = (_amount + leftover) / WEEK
```

This logic works for large reward deposits. However, if many small liquidations occur frequently:

- Rewards are split into many low-value _amount updates.
- The period_finish keeps increasing, stretching the reward duration.
- Users receive lower APY, and must wait longer to collect meaningful rewards.

Add a minimum threshold before calling deposit_reward_token() to avoid frequent low-value updates. And store amounts before threshold is reached.

[L-35] Missing minimum borrow amount check

The borrow() function does not enforce a minimum borrow amount, allowing users to open tiny debt positions. This can lead to many low-value debts, which are: - Gas-inefficient to liquidate, as each liquidation must handle a small position. - Operationally hard to track.

Add a minimum borrow threshold (e.g., 100 units) to prevent abuse.

[L-36] User can bypass `LendingPool`'s `borrowThreshold`

When a user borrow from the `LendinPool`, it will validate the debt not exceeding `borrowThreshold`.

```
    function _validateBorrow(address adapter, bytes calldata data, uint256 amount) internal
view {
        IAssetAdapter(adapter).validate(msg.sender, data);
        // check that the amount does not exceed borrow
        if (reserve.totalUsage + amount > parameters.borrowCap) revert BorrowCapReached();
        // For the borrow, we need to ensure that the user has enough collateral to cover the
new debt
        uint256 collateralValue = IAssetAdapter(adapter).getAssetValue(msg.sender, data);
        if (collateralValue == 0) revert NoCollateral();

        // We calculate the max debt that the user can have (based on the collateral value and
the borrow threshold)
>>>     uint256 maxDebt = collateralValue.percentMul(parameters.borrowThreshold);

        // We ensure that the position has enough collateral to cover the new debt or revert
```

```
        if (maxDebt < getPositionScaledDebt(adapter, msg.sender, data) + amount) {
            revert NotEnoughCollateralToBorrow();
        }
    }
```

This can be bypassed by calling `withdrawAsset` immediately after borrowing, as `withdrawAsset` only checks against `liquidationThreshold`.

```
    function _previewHealthFactor(uint256 collateralValue, uint256 scaledDebtValue) internal
view returns (uint256) {
        if (scaledDebtValue < 1) return WadRayMath.RAY;
>>>     uint256 collateralThreshold =
collateralValue.percentMul(parameters.liquidationThreshold);
        return (collateralThreshold * 1e18) / scaledDebtValue;
    }
```

Recommendations

Consider verifying `withdrawAsset` against `parameters.borrowThreshold` instead of `parameters.liquidationThreshold`.

[L-37] Without request timestamp validation stale data can overwrite

The `_processResponse` function in `RAACHousePriceOracle` processes responses based on `fulfillment` order without validating request timestamps:

```
function _processResponse (bytes32 requestId, bytes memory response) internal override {
    uint256 price = abi.decode(response, (uint256));
    uint256 houseId = requestToHouseId[requestId];
    housePrices.setHousePrice(houseId, price);
    delete requestToHouseId[requestId];
    emit HousePriceUpdated(houseId, price);
}
```

The contract only tracks the mapping from `requestId` to `houseId` but doesn't store `request timestamps`. This creates a race condition where:

- Request A is sent at 10:01 for houseId 123 (newer request).
- Request B is sent at 10:00 for houseId 123 (older request).
- Chainlink fulfills A first at 10:03 → sets price to 100.
- Chainlink fulfills B second at 10:04 → overwrites price to 95 (stale data).

The `RAACHousePrices` contract updates timestamps on every price set. However, this timestamp represents the fulfillment time, not the request time, making it impossible to determine which request was actually newer.

Note: This is the same for `RAACPrimeRateOracle`.

Reference: https://docs.chain.link/vrf/v2/security#use-requestid-to-match-randomness-requests-with-their-fulfillment-in-order.

Recommendations

Add request timestamp tracking to prevent stale data overwrites.

[L-38] Blacklisted users still able to transfer NFTs

In `RAACNFT`, the `notBlacklisted` modifier is applied to the `mint` function:

```
function mint(uint256 _tokenId, uint256 _amount) public override nonReentrant
notBlacklisted(msg.sender) { ... }
```

However, there is no similar check in the `transfer` logic (e.g., in `_update` or `transferFrom` functions). As a result, blacklisted users are still able to transfer or sell their NFTs, which defeats the purpose of the blacklist and may expose the protocol to regulatory or compliance risks.

Recommendations

Apply the `notBlacklisted` check (or equivalent logic) to all transfer-related functions.

[L-39] Missing staleness and sequencer checks in Chainlink price feeds

In `RAACHousePrices`, the following code is used to fetch price data:

```
(, int256 answer, , , ) = dataFeed.latestRoundData();
```

However, the contract does not check the returned `updatedAt` timestamp to ensure the price is fresh. This omission can result in the use of outdated or stale prices, especially if the oracle is not updating as expected.

Furthermore, for L2 chains (such as Optimism or Arbitrum), it is essential to check the sequencer status to avoid using prices when the sequencer is offline or the L2 is not in sync, which can lead to incorrect or manipulated price data.

Recommendations

Always check the `updatedAt` value from `latestRoundData()` to ensure the price is recent, if the price is not up-to-date, use the fallbackprice instead.

[L-40] `batchPriceUpdate()` in `RAACHousePrices` contract is unreachable

In `RAACHousePrices`, both of the following functions are marked `onlyOracle`:

```
function setHousePrice(uint256 _tokenId, uint256 _amount) external onlyOracle { ... }
function setHousePrices(uint256[] calldata tokenIds, uint256[] calldata amounts) external
onlyOracle { ... }
```

However, in the current implementation of `RAACHousePriceOracle`, only `setHousePrice` is called:

```
housePrices.setHousePrice(houseId, price);
```

There is no code path that calls `setHousePrices`, making the batch update feature unreachable and non-functional.

Recommendations

Either implement batch price update logic in the oracle contract to utilize `setHousePrices`, or remove the unused function from `RAACHousePrices` to avoid confusion and reduce code complexity.

[L-41] Stability Pool does not account for bad debt after liquidation

The Stability Pool allows users to deposit rToken and mint deToken at a 1:1 ratio, using the LendingPool's normalized index to account for accrued interest. During a liquidation, the Stability Pool transfers rToken to cover a borrower's debt and receives the underlying asset (e.g., ERC20 or NFT) in return and deposits back rTokens.

However, if a liquidation results in partial recovery (i.e., bad debt), then the pool's rToken balance decreases without fully matching compensation. This loss is not distributed proportionally among deToken holders. Instead, users are still allowed to withdraw based on their individual scaled balances, assuming full backing, which may no longer be true.

This creates a vulnerability where:

- The first users to withdraw exit with their full shares.
- Remaining users absorb the loss.
- The last user(s) may be unable to withdraw at all.

Recommendations

Replace the 1:1 rToken <-> deToken redemption logic with a proportional withdrawal mechanism based on current pool backing.

[L-42] Incompatible oracle interface leads to adapter failures`

The ERC721VaultAdapter and ERC721AssetAdapter contracts call priceOracle.getLatestPrice(tokenId), expecting a single uint256 price value. However, the actual implementation in RAACHousePrices returns two values:

`function getLatestPrice(uint256 _tokenId) external view returns (uint256 _crvUSDPrice, uint256 _lastUpdateTimestamp);` This mismatch causes any function that relies on getLatestPrice() (like deposit() or getWithdrawValue()) to revert.

Recommendations

Update the adapter contracts to correctly handle tuple unpacking. Or, update the oracle to return only the price if the timestamp is not needed.